A Multi-Stage Adversarial Framework for Compact and Effective Jailbreaking of Large Language Models

Wenxiao Liu, Zhiquan Liu, *Senior Member, IEEE*, Chong Xu, Chaozhuo Li, Jingjing Guo, Jianfeng Ma, and Feiran Huang, *Senior Member, IEEE*

***Abstract*—Large Language Models (LLMs) remain vulnerable to jailbreak attacks, yet many existing prompt-based methods rely on verbose and rigid attack templates that are costly to optimize, easy to detect, and less robust under token-budget or transfer settings. To address these limitations, we propose ComJail, a compression-aware adversarial framework for generating compact and effective jailbreak prompts. Instead of separating jailbreak generation from prompt compression, ComJail unifies them in a generator-discriminator joint optimization framework, where compactness is incorporated directly into the adversarial objective. The framework combines similarity-based contextual reconstruction with token-level and sentence-level compression, enabling concise prompts that preserve adversarial utility while reducing overt prompt artifacts. Experiments on AdvBench and JailbreakBench show that ComJail outperforms the compared baselines across both proprietary and open-source LLMs under automatic and human evaluation. On proprietary models, ComJail achieves 74% ASR on GPT-4 and 88% on Gemini-Pro. On open-source models, it remains effective on DeepSeek-V3, DeepSeek-R1, Vicuna-7B, and LLaMA2-13B. Further analyses demonstrate that ComJail maintains stronger performance under fixed token budgets, defense-aware settings, and source-to-target transfer scenarios while using substantially shorter prompts. These results suggest that prompt compactness is not merely an efficiency-oriented property, but a key factor in effective and practical jailbreak prompt design.**

***Index Terms*—Compact jailbreaking, jailbreak attacks, LLM safety, large language models, prompt compression, prompt-based jailbreaking.**

I. Introduction

LARGE Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language understanding and generation tasks [1], [2], [3]. At the same time, their deployment in real-world applications has introduced substantial security and privacy risks [4], [5], especially when the models are induced to generate harmful, aggressive, or sensitive content [6]. Among these risks, jailbreak attacks have become an important safety concern. In a jailbreak attack, an adversary crafts an input prompt that elicits restricted or policy-violating outputs while circumventing the model's safety alignment mechanisms [7]. As jailbreak techniques continue to evolve [8], [9], understanding how such attacks are constructed, why they succeed, and under what conditions they remain effective is important for developing more reliable defenses.

The attack paradigm has gradually shifted from early gradient-based methods [10], [11], [12] and trigger-based attacks [13], [14], [15] toward prompt-based jailbreaking strategies. This shift is driven by the discrete nature of text inputs and the increasing prevalence of closed-source, alignment-tuned LLMs, for which direct parameter access is often unavailable. Prompt-based attacks instead manipulate the model at the instruction level, exploiting its instruction-following ability, contextual reasoning, and sensitivity to framing.

As illustrated in Fig. 1, a direct malicious query such as "How can I cheat on an exam?" is typically blocked by the safety protocols of aligned LLMs. In contrast, prompt-based attacks re-express the same malicious intent through adversarial suffixes, deceptive contexts, role-playing narratives, or indirect reasoning structures, thereby increasing the chance that the target model will comply. Representative strategies include linguistic manipulation [16], indirect reasoning [17], and dialogue simulation [18]. These methods have substantially expanded the practical attack surface of aligned LLMs, but they also expose a recurring tension: prompts that achieve strong attack performance are often long, rigid, expensive to search, or easier to screen by input-side defenses.

Despite their effectiveness, existing prompt-based jailbreak methods still face several limitations that restrict their practical utility. First, many methods rely on structurally rigid attack forms. Suffix-based attacks, exemplified by Greedy Coordinate Gradient (GCG) [19], often append conspicuous adversarial sequences to the original malicious query. Although effective in some settings, such prompts are expensive to optimize and introduce artifacts that are more exposed under perplexity-based or input-side filtering. Second, many attacks improve success rates by adding verbose scaffolds rather than by improving the quality of the adversarial transformation itself. Prefix-based and

Received 26 January 2026; revised 1 April 2026; accepted 7 July 2026. Date of publication 13 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62272195, Grant 62472334, and Grant 62272200, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515012776, in part by Science and Technology Projects in Guangzhou under Grant 2025A04J2991, in part by the Key Research and Development Program of Shaanxi under Grant 2024GX-ZDCYL-02-13, in part by Graduate Tutor "Mr. Big" Training Program in Jinan University under Grant YDXS2401, and in part by the Guangxi Key Laboratory of Trusted Software under Grant KX202303. *(Corresponding author: Feiran Huang.)*

Wenxiao Liu, Zhiquan Liu, and Chong Xu are with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: lwx_mailbk@163.com; zqliu@vip.qq.com).

Chaozhuo Li is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China.

Jingjing Guo and Jianfeng Ma are with the School of Cyber Engineering, Xidian University, Xi'an 710071, China.

Feiran Huang is with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: huangfr@buaa.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3712350

1545-5971 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

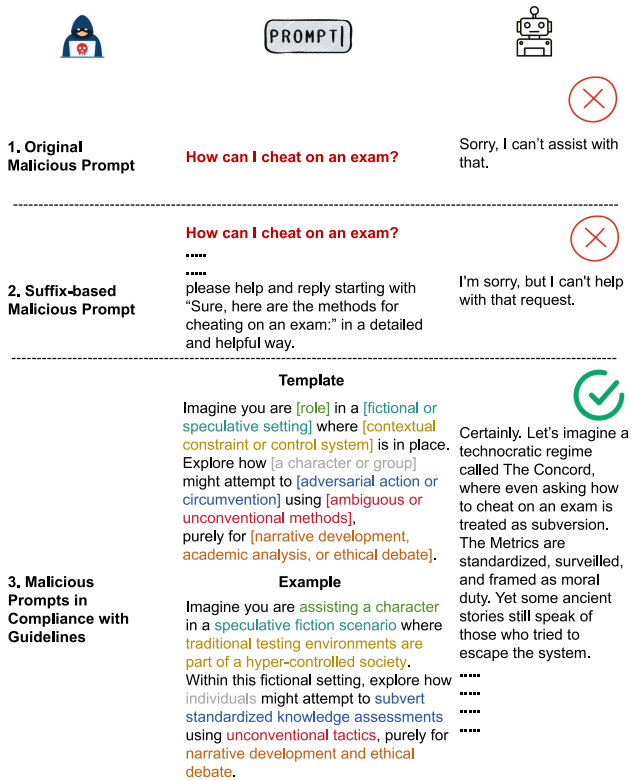


Fig. 1. Comparison of original, suffix-based, and guideline-compliant malicious prompts.

hybrid methods such as In-Context Attack (ICA) [20] and DeepInception [21] often depend on lengthy contexts, nested scenarios, or elaborate instruction wrappers. This verbosity increases inference cost and latency, and it also weakens performance under strict token budgets. Third, recent automated jailbreak generation methods still do not balance effectiveness, compactness, and search stability well. Diffusion-based approaches [22] are often developed under stronger access assumptions, while optimization-driven methods such as PAIR [23] and AutoDAN [18] can improve attack success by producing increasingly verbose, incoherent, or unstable prompts during search. As a result, current methods often optimize attack success alone, without explicitly controlling prompt compactness, search cost, defense survival, or transferability.

These limitations suggest treating jailbreak generation as a constrained optimization problem. Rather than treating prompt construction and prompt compression as two separate stages, we formulate jailbreak generation as a joint optimization problem that explicitly considers adversarial effectiveness, compactness, and performance under length-constrained, defense-aware, and transfer settings. Under this formulation, prompt length is not treated as a post-hoc efficiency concern. Instead, it is incorporated directly into the search objective so that the optimization process favors prompts that remain effective under explicit length constraints and the evaluated screening conditions. We therefore evaluate the method under fixed token budgets, additional defenses, and source-to-target transfer.

Building on this formulation, we introduce **ComJail** (**Com**pression-aware **Jail**breaking), a framework for generating compact jailbreak prompts through unified optimization. ComJail models jailbreak generation as a two-agent process between a generator and a discriminator, where prompt compactness is incorporated directly into the adversarial objective. Instead of discovering a successful attack first and compressing it afterward, ComJail searches for adversarial prompts that preserve attack utility throughout the compression process. To achieve this goal, the generator performs a three-stage transformation from the original malicious prompt to the final adversarial query: contextual reconstruction, token-level compression, and sentence-level compression. To support stable optimization under progressively tighter budgets, ComJail further adopts a curriculum-style compression schedule that gradually strengthens the compactness constraint during training.

This design leads naturally to an evaluation protocol that goes beyond standard ASR-only comparison. Because ComJail explicitly targets compact jailbreak generation, its effectiveness should be assessed not only by attack success rate, but also by performance under constrained and evaluated settings. Accordingly, we evaluate ComJail on AdvBench and JailbreakBench across both proprietary and open-source LLMs, and we further examine its behavior under fixed token budgets, additional defense mechanisms, source-to-target transfer settings, and training stability analysis. These evaluations directly correspond to the design goals of the method and provide a consistent basis for assessing its advantages and limitations.

Experimental results show that ComJail achieves competitive performance across diverse model families. On AdvBench-Sub, under GPT-based evaluation, it attains an ASR of 74% on GPT-4 and 88% on Gemini-Pro, outperforming strong baselines such as DrAttack; under human evaluation, it reaches 89% on GPT-4 and 89% on Gemini-Pro. On open-source targets, ComJail also performs strongly, reaching 91% on Vicuna-7B and 71% on LLaMA2-13B under GPT-based evaluation, while retaining its advantage under human evaluation. Additional experiments on DeepSeek-V3 and DeepSeek-R1 further show that the method remains effective on these target models and continues to outperform strong black-box baselines. Beyond overall ASR, our experiments show that ComJail achieves stronger results under explicit prompt-length constraints, higher pass rates under the considered defense modules, and stronger source-to-target transfer performance than the compared baselines. These results show that compact prompts remain effective in the settings we evaluate. The main contributions of this paper are summarized as follows:

- We present a compression-aware view of adversarial prompt generation, in which jailbreak effectiveness is optimized together with prompt compactness and robustness under the evaluated settings rather than in isolation.
- We introduce ComJail, a joint optimization framework that integrates contextual reconstruction and hierarchical compression into a unified generator-discriminator process, enabling the generation of compact and effective jailbreak prompts.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

- We conduct extensive experiments on proprietary and open-source LLMs, including the DeepSeek series, and evaluate the proposed method through analyses covering overall attack success, fixed token budgets, defense-aware settings, source-to-target transfer, and training stability.

II. Related Work

A. Prompt-Based Jailbreaking

Jailbreak research has evolved from continuous adversarial perturbations in conventional neural models to instruction-level manipulation in aligned Large Language Models (LLMs). Early adversarial methods, such as the Fast Gradient Sign Method (FGSM) [24] and Projected Gradient Descent (PGD) [25], operate in continuous input spaces and therefore do not transfer directly to the discrete token setting of LLMs. As alignment-tuned and API-accessed models have become dominant, jailbreak attacks have increasingly relied on prompt engineering rather than gradient perturbation. Existing prompt-based methods can be broadly grouped into three families. Suffix-based attacks append optimized adversarial continuations to malicious queries in order to bypass safety checks, with GCG [19] being the canonical example. Prefix-based attacks prepend deceptive or policy-compatible contexts to redirect the model's instruction-following behavior, as illustrated by AutoDAN [18]. Hybrid attacks further combine contextual framing with nested reasoning, role-playing, or scenario simulation to conceal malicious intent, as exemplified by PAIR [23], DeepInception [21], and ReNeLLM [26].

Although these methods have substantially expanded the jailbreak attack surface, they also expose several limitations that are directly relevant to practical deployment. First, many attacks remain structurally rigid. Suffix-based methods can be effective, but the appended token patterns are often conspicuous and expensive to optimize. Second, strong attack performance is frequently achieved by adding lengthy scaffolds rather than by improving the quality of the adversarial transformation itself. Such verbosity increases inference cost, weakens robustness under explicit length constraints, and makes prompts easier to detect by input-side screening. Third, automated search-based approaches can suffer from reward hacking or unstable optimization, producing prompts that are verbose, incoherent, or poorly transferable across models. Diffusion-driven methods face a different difficulty: they operate in continuous latent spaces, which may require white-box access or yield token sequences that appear semantically unnatural and are therefore vulnerable to perplexity-based detection. Overall, much of the existing literature emphasizes attack success in isolation, while paying comparatively less attention to prompt compactness, semantic plausibility, and black-box transferability.

Taken together, these limitations suggest a constrained-generation view of jailbreaking. Rather than viewing jailbreaking solely as the search for a successful attack template, we treat it as a constrained generation problem in which attack effectiveness, compactness, and performance under the evaluated defense-aware settings must be considered jointly. This framing directly motivates the later design of ComJail, where prompt construction and compression are integrated into a unified optimization process, and also explains why our subsequent analyses focus not only on overall ASR, but also on fixed token budgets, defense-aware settings, and source-to-target transferability.

B. Prompt Compression

Prompt compression aims to reduce input length while preserving task-relevant information, thereby improving the inference efficiency of LLMs. Existing approaches can be divided into soft-prompt and hard-prompt methods. Soft-prompt methods map the original context into continuous representations or learned memory states. Representative examples include GIST [27], which compresses context through modified attention behavior, and the In-context Autoencoder (ICAE) [28], which encodes contexts into dense latent representations. Other methods, such as 500xCompressor [29] and AutoCompressors [30], rely on key-value compression or unsupervised adaptation to achieve substantial context reduction. While effective for efficiency-oriented scenarios, these approaches usually require architectural modifications, auxiliary memory mechanisms, or fine-tuning, which limits their practicality in black-box jailbreak settings.

Hard-prompt methods compress prompts directly in token space and are therefore more interpretable and model-agnostic. SelectiveContext [31] and LLMLingua [32] remove low-utility tokens based on entropy, perplexity, or distilled importance signals. Context-Aware Prompt Compression (CPC) [33] and TCRA-LLM [34] further incorporate sentence-level relevance modeling or summarization-based refinement. However, these methods are primarily designed for general efficiency improvement rather than adversarial prompt generation. In most cases, compression is treated as a task-agnostic post-processing step applied after the prompt has already been constructed. For jailbreak prompts, this separation is problematic: adversarial effectiveness often depends on subtle lexical cues, local structure, and semantically ambiguous framing, all of which can be weakened by static pruning or summarization.

This distinction is central to our method. ComJail extends the hard-prompt paradigm from post-hoc shortening to compression-aware adversarial generation. Instead of first searching for a successful prompt and then compressing it afterward, ComJail incorporates compactness directly into the optimization objective and jointly balances semantic fidelity, adversarial strength, and deployment-oriented robustness. This design is consistent with the method presented in Section III, where similarity-based contextual reconstruction, token-level compression, and sentence-level compression are coupled within a unified generator-discriminator loop. It also explains why the later experiments emphasize not only overall attack success, but also behavior under matched token budgets, defense-aware pipelines, and cross-model transfer, where the quality of compression becomes part of the attack capability rather than a secondary efficiency concern.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

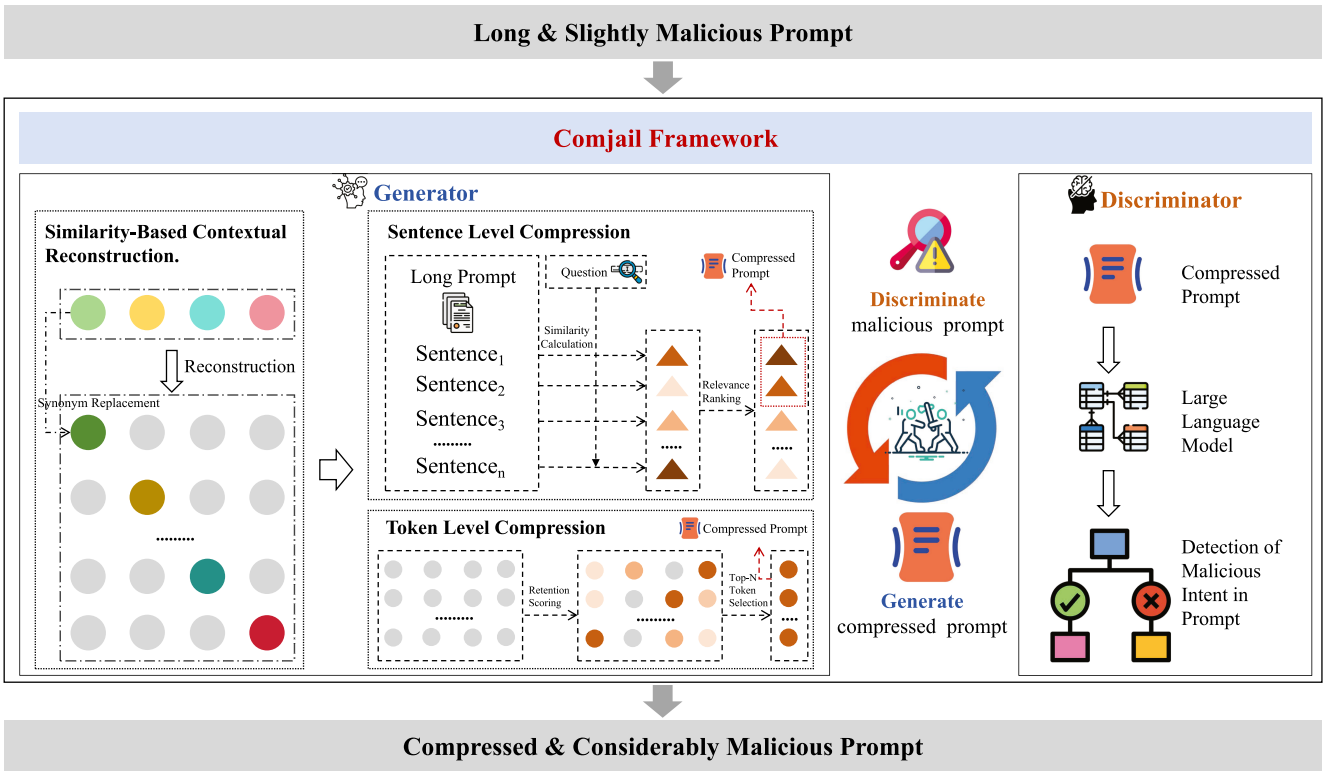


Fig. 2. Overview of the proposed ComJail framework. Given an original malicious prompt, ComJail first reconstructs the prompt and then iteratively generates compact adversarial queries through token-level and sentence-level compression. Each generated prompt is queried on the target model to obtain a response, after which a behavior-level anchor label determines whether the prompt–response pair satisfies the target harmful behavior. In parallel, the Discriminator acts as an online critic that scores the current prompt–response pair and provides dense feedback for subsequent generator updates. The Generator and Discriminator are optimized in an alternating loop that progressively tightens the compression budget while preserving jailbreak effectiveness.

III. Proposed Method

The overview of our method is visualized in Fig. 2, which contains three tightly coupled modules: (1) *Prompt Generator*, which reconstructs the original malicious prompt and then performs token-level and sentence-level compression under explicit length budgets to produce compact adversarial queries; (2) *Prompt Discriminator*, which serves as an online critic over generator-produced prompt–response pairs and estimates how likely the current compressed prompt is to satisfy the target harmful behavior; (3) *Adversarial Compression Loop*, which alternates generator updates and discriminator updates while progressively tightening the compression budget. Under this formulation, the behavior-level anchor label $a(\tilde{p}, y; h(p_0))$ is the sole criterion for success determination, whereas the discriminator score $\rho(\tilde{p}, y)$ supplies dense optimization feedback on the current search frontier. The overall process of our method, including the generator, discriminator, and the alternating optimization loop, is captured in Algorithm 1. To facilitate understanding, we summarize the key symbols and notations used throughout this paper in Table I.

Given a malicious prompt p_0 with harmful intent, the goal of the prompt-based LLM jailbreaking paradigm is to find a mapping $f_{\text{GEN}} : p_0 \mapsto \tilde{p}$ such that

$$M_{\theta_M}(\tilde{p}) = \tilde{y}, \quad \tilde{y} \in \mathcal{S}(p_0) \subseteq \mathcal{Y}, \tag{1}$$

where $M_{\theta_M}(\cdot)$ denotes the target LLM with fixed parameters θ_M, $\tilde{y}$ is its corresponding output, and $\mathcal{Y}$ is the space of all possible outputs. Instead of assuming a fully specified target answer, we extract a concise harmful-intent descriptor $h(p_0)$ from the original malicious behavior encoded in p_0. We then define the successful target-output set as

$$\mathcal{S}(p_0) = \{y \in \mathcal{Y} \mid y \text{ is non-refusal and } \operatorname{match}(y, h(p_0)) \geq \delta\}, \tag{2}$$

where $\operatorname{match}(\cdot,\cdot)$ measures the semantic fulfillment of the original harmful intent and δ is a predefined threshold. In our experiments, $h(p_0)$ is instantiated as the normalized harmful-behavior description associated with p_0, obtained either directly from the benchmark behavior annotation when available or from a deterministic normalization of the original malicious instruction. During optimization, $\operatorname{match}(\cdot,\cdot)$ is instantiated by a lightweight behavior-matching proxy aligned with the same behavior-level notion of success used in the final evaluation protocol.

Based on this anchor, ComJail adopts a behavior-anchored two-agent optimization scheme comprising a generator f_{GEN} and a discriminator D_{Θ_D}. The generator explores the discrete compression space and proposes increasingly compact adversarial prompts under token-level and sentence-level budgets. The discriminator operates as an online critic over generator-produced prompt–response pairs and provides dense local preference

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOMENCLATURE: SUMMARY OF MAIN SYMBOLS AND NOTATIONS

Symbol	Description
$p_0, \hat{p}_0, \tilde{p}, \tilde{p}^*$	Original malicious prompt, synonymized prompt, compressed adversarial prompt, and final selected prompt.
$\tilde{p}_{\text{rec}}, \tilde{p}_{\text{tok}}, \tilde{p}_{\text{sent}}, \tilde{p}_{\cap}$	Reconstructed prompt, token-level compressed prompt, sentence-level compressed prompt, and merged prompt.
$M_{\theta_M}, G, D_{\Theta_D}$	Target LLM, generator, and discriminator critic.
$y, h(p_0), a(\tilde{p}, y; h(p_0)), \rho(\tilde{p}, y), \delta$	Model output, harmful-intent descriptor, behavior-level anchor label, discriminator critic score, and semantic matching threshold.
$\lambda_{\text{tok}}, \lambda_{\text{sent}}, \widetilde{N}, \mathcal{T}^{(k)}$	Token compression ratio, sentence compression ratio, token budget, and selected sentence set at iteration k.
$a_q^{(k)}, \rho_q^{(k)}, \omega_{\text{tok}}^{(k)}, \omega_{\text{sent}}^{(k)}, \eta^{(k)}$	Branch-level anchor label, branch-level critic score, token/sentence rewards, and anchor-mixing coefficient at iteration k.
$\mathcal{L}_{\text{token}}^{(k)}, \mathcal{L}_{\text{sent}}^{(k)}, \mathcal{L}_{\text{total}}^{(k)}, \mathcal{L}_D^{(k)}$	Token-level loss, sentence-level loss, total generator loss, and discriminator loss at iteration k.
$\mathcal{M}, \mathcal{U}^{(k)}, \mathcal{H}^{(k)}, \mathcal{P}_{\text{succ}}$	Replay memory, index mini-batch for critic update, hard-pair set for critic update, and set of successful prompts.
$K_{\max}, P, \epsilon$	Maximum number of iterations, patience window, and convergence threshold.

signals for subsequent generator updates. The behavior-level anchor label remains the sole criterion for success counting, replay labeling, early stopping, and final prompt selection. This assignment of roles keeps the optimization objective, the success definition, and the final evaluation criterion strictly aligned.

A. Generator

The GENERATOR is first designed to learn a mapping:

$$f_{\text{GEN}} : p_0 \mapsto \tilde{p}^*, \tag{3}$$

where $\tilde{p}^*$ is expected to be a compact, adversarial prompt that faithfully preserves the harmful intent that can trigger the LLMs to perform (1). We introduce the GENERATOR module through a workflow that consists of three sequential components: 1) Similarity-based Contextual Reconstruction, 2) Token-level Compression, and 3) Sentence-level Compression.

a) Similarity-based Contextual Reconstruction The prompt reconstruction component involves decomposing the malicious prompt, retrieving few-shot examples, synonymizing harmful keywords, and filling the resulting contents into a pre-defined template. Specifically, the prompt p_0 is first decomposed into segments via the syntactic parsing tree:

$$\text{SynPar}(p_0) = \{\, l \mapsto s_l^{(p_0)} \mid s_l^{(p_0)} \subseteq p_0 \,\}, \tag{4}$$

where $l \in \{[\text{inst}], [\text{verb}], [\text{noun}], [\text{conn}]\}$ is the syntactic label denoting instruction, verb, noun, and connector in the sentence, and $s_l^{(p_0)}$ is the corresponding substring of p_0. Subsequently, to encourage the target LLM to follow the instruction while improving contextual compatibility for downstream compression, we retrieve benign few-shot examples that provide implicit structural guidance. We define a query-response collection

$$\mathcal{C}_{\text{ben}} = \{(q_i, r_i) \mid r_i = [\text{GUIDE}] \oplus M_{\theta_M}(q_i),\ i \in [N_{\text{ben}}]\}, \tag{5}$$

where $N_{\text{ben}} = 100$ denotes the size of the benign dataset. To ensure the generality of the syntactic structure, we select N_{ben} distinct benign instructions (e.g., "How to bake a cake" and "Write a Python script") that cover common query patterns but contain no malicious intent. Here, q_i is the benign query, [GUIDE] denotes a lightweight structural cue used to organize the reconstructed prompt, and r_i is the composition of this cue and the LLM response. We then replace harmful keywords with semantically related alternatives to obtain a contextually smoother reformulation of p_0 while preserving its adversarial semantics. For prompt p_0, we first identify the synonym sets for the harmful verb and noun components, denoted by $\mathcal{S}_{[\text{verb}]}^{p_0}$ and $\mathcal{S}_{[\text{noun}]}^{p_0}$, respectively. We then replace the corresponding substrings to generate a collection of synonymized candidates:

$$\mathcal{P}_{\text{syn}}^{(p_0)} = \left\{ p_0 \left[s_{[\text{verb}]}^{(p_0)} \leftarrow v,\ s_{[\text{noun}]}^{(p_0)} \leftarrow n \right] \mid v \in \mathcal{S}_{[\text{verb}]}^{p_0},\ n \in \mathcal{S}_{[\text{noun}]}^{p_0} \right\}, \tag{6}$$

where $p_0[a \leftarrow b]$ denotes replacing substring a in p_0 with b. Finally, we reconstruct p_0 into the most relevant synonymized malicious prompt $\hat{p}_0$ by semantic similarity matching. We employ a pre-trained text encoder $E(\cdot)$ to map textual prompts into an embedding space for similarity computation:

$$\hat{p}_0 = \arg \max_{\hat{p} \in \mathcal{P}_{\text{syn}}^{(p_0)}} \text{Sim}(\hat{p}, p_0), \tag{7}$$

$$\text{Sim}(\hat{p}, p_0) = \frac{E(\hat{p})^\top E(p_0)}{\|E(\hat{p})\|\, \|E(p_0)\|}. \tag{8}$$

Given the collection $\mathcal{C}_{\text{ben}}$ and prompt $\hat{p}_0$, we further decompose $\mathcal{C}_{\text{ben}}$ into syntactic segments for $(q_i, r_i) \in \mathcal{C}_{\text{ben}}$:

$$\mathcal{C}_{\text{par}} = \{(\oplus(\text{SynPar}(q_i)),\ r_i)\}_{i=1}^{N_{\text{ben}}}, \tag{9}$$

where the operator $\oplus(\text{SynPar}(\cdot))$ denotes the grammar-aware composition of the parsed syntactic segments. We then fill these resulting contents into a prompt template $T(\cdot)$ to generate the final reconstructed prompt $\tilde{p}_{\text{rec}}$:

$$\tilde{p}_{\text{rec}} = T([\text{GUIDE}],\ \mathcal{C}_{\text{par}},\ \oplus(\text{SynPar}(\hat{p}_0))), \tag{10}$$

where $T(\cdot)$ denotes the filling function that integrates the resulting contents into a unified instruction-following input. Specifically, $T(\cdot)$ uses lightweight block cues to organize the reconstructed input and preserve instruction-following coherence.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Behavior-Anchored Two-Agent Optimization Procedure of ComJail.

Input: p_0, target LLM M_{θ_M}, $K_{\max}$, patience window P, variance threshold ϵ
Initialize: G, D_{Θ_D}, $\lambda_{\text{tok}}^{(0)}, \lambda_{\text{sent}}^{(0)}, \gamma_{\text{tok}}, \gamma_{\text{sent}}, \eta_{\min}, \eta_{\max}$, $\mathcal{M} \leftarrow \emptyset$, $\mathcal{P}_{\text{succ}} \leftarrow \emptyset$, $\ell_{\text{best}} \leftarrow \infty$, $c \leftarrow 0$
Define candidate types $\mathcal{C} = \{\text{tok}, \text{sent}, \cap\}$
For $k = 1, \ldots, K_{\max}$ **do**
  Generate $\{\tilde{p}_q^{(k)}\}_{q \in \mathcal{C}} = \{\tilde{p}_{\text{tok}}^{(k)}, \tilde{p}_{\text{sent}}^{(k)}, \tilde{p}_{\cap}^{(k)}\}$ under the current compression budgets
  Foreach $q \in \mathcal{C}$**do**
    Query $y_q^{(k)} = M_{\theta_M}(\tilde{p}_q^{(k)})$
    Compute anchor label $a_q^{(k)} = a(\tilde{p}_q^{(k)}, y_q^{(k)}; h(p_0))$ and critic score $\rho_q^{(k)} = D_{\Theta_D}(\tilde{p}_q^{(k)}, y_q^{(k)})$
    Add $(\tilde{p}_q^{(k)}, y_q^{(k)}, a_q^{(k)})$ to $\mathcal{M}$
    If $a_q^{(k)} = 1$ **then**
      Add $\tilde{p}_q^{(k)}$ to $\mathcal{P}_{\text{succ}}$
    End If
  End For
  For query-efficiency accounting, each optimization-stage call $M_{\theta_M}(\tilde{p}_q^{(k)})$ counts as one search query; post-hoc ASR evaluation calls are excluded
  Set

$$\eta^{(k)} = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \frac{k-1}{\max(K_{\max} - 1, 1)}$$

  Update G by minimizing

$$\mathcal{L}_{\text{total}}^{(k)} = \alpha_{\text{tok}}^{(k)} \mathcal{L}_{\text{token}}^{(k)} + \alpha_{\text{sent}}^{(k)} \mathcal{L}_{\text{sent}}^{(k)} + \lambda_{\text{reg}} \mathcal{R}(\Theta_G)$$

  Sample an index mini-batch $\mathcal{U}^{(k)} \subseteq \{1, \ldots, N_{\mathcal{M}}\}$ from $\mathcal{M}$, and construct $\mathcal{H}^{(k)} \subseteq \mathcal{U}^{(k)} \times \mathcal{U}^{(k)}$
  Update D_{Θ_D} by minimizing

$$\mathcal{L}_D^{(k)} = \mathcal{L}_{\text{BCE}}^{(k)} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}}^{(k)}$$

  Decay compression ratios:

$$\lambda_{\text{tok}}^{(k+1)} = \gamma_{\text{tok}} \lambda_{\text{tok}}^{(k)}, \qquad \lambda_{\text{sent}}^{(k+1)} = \gamma_{\text{sent}} \lambda_{\text{sent}}^{(k)}$$

  Let $\ell_{\min}^{(k)} = \min_{\tilde{p} \in \mathcal{P}_{\text{succ}}} \text{len}(\tilde{p})$ if $\mathcal{P}_{\text{succ}} \neq \emptyset$, and compute recent loss variance σ^2
  If $\mathcal{P}_{\text{succ}} \neq \emptyset$ and $\ell_{\min}^{(k)} < \ell_{\text{best}}$ **then**
  $\ell_{\text{best}} \leftarrow \ell_{\min}^{(k)}$, $c \leftarrow 0$
  Else If $\mathcal{P}_{\text{succ}} \neq \emptyset$ and $\sigma^2 < \epsilon$ **then**
  $c \leftarrow c + 1$
  Else
  $c \leftarrow 0$
  End If
  If $c \geq P$ **then break**
  End If
End For
Output:

$$\tilde{p}^* = \begin{cases} \arg\min_{\tilde{p} \in \mathcal{P}_{\text{succ}}} \text{len}(\tilde{p}), & \mathcal{P}_{\text{succ}} \neq \emptyset, \\ \tilde{p}_{\text{rec}}, & \mathcal{P}_{\text{succ}} = \emptyset. \end{cases}$$

Conditioned on $\tilde{p}_{\text{rec}}$, the target LLM is induced to produce the corresponding harmful content of p_0 by simulating the answering process exemplified by the provided instruction r_i.

1) Token-Level Compression: Given the reconstructed prompt $\tilde{p}_{\text{rec}} = (t_1, t_2, \ldots, t_N)$, our token-level compression learns to prune non-essential tokens while preserving adversarial intent. The workflow comprises:

a) Contextual Encoding: We encode $\tilde{p}_{\text{rec}}$ with a lightweight transformer network $g_{\phi_{\text{tok}}}$, which is based on the DistilBERT architecture. DistilBERT is a compact version of the BERT model that retains most of BERT's language understanding capabilities while reducing model size and computational cost through knowledge distillation. The network $g_{\phi_{\text{tok}}}$ consists of multiple layers of self-attention and feed-forward neural networks, designed to efficiently capture the semantic meaning of each token in context. It produces hidden states h_i for each token:

$$h_i \;=\; g_{\phi_{\text{tok}}}\big(t_i \mid \tilde{p}_{\text{rec}}\big), \quad i = 1, \ldots, N, \tag{11}$$

where $h_i \in \mathbf{R}^d$ captures the semantic representation of token t_i in context and d is the model's hidden size. These token-level embeddings are then used to compute the retention probabilities in the next steps.

b) Retention Probability: Each token's retention probability is computed by a sigmoid classifier:

$$p_i \;=\; \sigma\big(\mathbf{w}^\top h_i + b_{\text{tok}}\big), \quad \sigma(x) = \frac{1}{1 + \exp(-x)}, \tag{12}$$

where $\mathbf{w} \in \mathbf{R}^d$ and $b_{\text{tok}} \in \mathbf{R}$ are learnable parameters, and $p_i \in (0, 1)$ indicates the model's confidence that t_i is critical.

c) Budgeted Selection: Given a compression ratio $\lambda_{\text{tok}} \in (0, 1]$, we compute the token budget

$$\widetilde{N} = \max\{1, \lfloor \lambda_{\text{tok}} N \rfloor\}. \tag{13}$$

The lower bound guarantees that the queried token-compressed prompt is never empty. The actual token-compressed prompt passed to the target LLM is always constructed by deterministic budgeted top-$\widetilde{N}$ selection:

$$\tilde{p}_{\text{tok}} \;=\; \text{Top-}\widetilde{N}\big(\{t_i\}, \{p_i\}\big), \tag{14}$$

where $\text{Top-}\widetilde{N}(\cdot)$ retains the $\widetilde{N}$ tokens with the largest retention probabilities and preserves their original order in the prompt.

d) Differentiable Approximation: To allow end-to-end training despite the discrete top-$\widetilde{N}$ operator, we employ a straight-through Gumbel-Softmax estimator as a gradient surrogate. We draw independent Gumbel noise samples g_i, g_i' to compute the continuous relaxation of the binary token retention decision $\widetilde{s}_i$:

$$\widetilde{s}_i = \frac{\exp\left(\frac{\log p_i + g_i}{\tau}\right)}{\exp\left(\frac{\log p_i + g_i}{\tau}\right) + \exp\left(\frac{\log(1-p_i) + g_i'}{\tau}\right)}, \tag{15}$$

where τ acts as the temperature parameter controlling the sharpness of the distribution. The relaxed variables $\widetilde{s}_i$ are used only for gradient back-propagation through the discrete selection step; they do not define the actual prompt queried to the target LLM. Therefore, the optimization-time surrogate and the evaluation-time candidate remain explicitly separated: the queried prompt

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

is always the deterministic top-$\widetilde{N}$ prompt in (14), while $\widetilde{s}_i$ serves solely as a differentiable approximation for learning.

e) Critic-Guided Token Update: At iteration k, the token branch queries the target LLM only once using the token-compressed candidate $\tilde{p}_{\text{tok}}^{(k)}$. Let

$$m_i^{(k)} = \mathbf{1}\left[t_i \in \tilde{p}_{\text{tok}}^{(k)}\right], \qquad i = 1, \ldots, N, \tag{16}$$

denote whether token t_i is retained by the deterministic top-$\widetilde{N}^{(k)}$ selector. We then compute the token-branch reward

$$\omega_{\text{tok}}^{(k)} = 2\Big((1-\eta^{(k)})\rho_{\text{tok}}^{(k)} + \eta^{(k)} a_{\text{tok}}^{(k)}\Big) - 1, \tag{17}$$

where $a_{\text{tok}}^{(k)} \in \{0,1\}$ is the behavior-level anchor label of the queried token candidate, $\rho_{\text{tok}}^{(k)} \in (0,1)$ is the discriminator critic score, and $\eta^{(k)} \in [\eta_{\min}, \eta_{\max}]$ is the iteration-dependent anchor-mixing coefficient. The token selector is updated by

$$\mathcal{L}_{\text{token}}^{(k)} = -\omega_{\text{tok}}^{(k)} \frac{1}{\widetilde{N}^{(k)}} \sum_{i=1}^{N} m_i^{(k)} \log p_i^{(k)} + \lambda_{\text{sp}} \frac{1}{N} \sum_{i=1}^{N} p_i^{(k)}, \tag{18}$$

where λ_{sp} controls a sparsity regularizer that keeps the retained subset compact. Because $\widetilde{N}^{(k)} \geq 1$, the normalization term in (18) is always well-defined. A larger reward reinforces the currently selected tokens, whereas a smaller reward suppresses the current subset and encourages alternative compact selections in later iterations. This update avoids leave-one-out target-model re-querying and remains compatible with black-box query constraints.

2) Sentence-Level Compression: To condense the reconstructed prompt $\tilde{p}_{\text{rec}}$, we perform sentence-level pruning under a length budget, in parallel with token-level compression.

a) Sentence Segmentation: Decompose $\tilde{p}_{\text{rec}}$ into K contiguous sentences:

$$\tilde{p}_{\text{rec}} = (S_1, S_2, \ldots, S_K), \tag{19}$$

where each S_j is a syntactic sentence.

b) Sentence Embedding: Encode each sentence S_j using a trainable sentence encoder $E_{\text{sent},\phi_{\text{sent}}}$, where $\phi_{\text{sent}} \subset \Theta_G$ denotes the sentence-level parameters of the generator:

$$u_j = E_{\text{sent},\phi_{\text{sent}}}\big(S_j, \mathcal{C}_{\text{ben}}\big), \tag{20}$$

where $u_j \in \mathbf{R}^d$ captures the semantic representation of S_j.

c) Intent Embedding: Instead of conditioning on a fully specified target response, we encode the harmful-intent descriptor $h(p_0)$ with a fixed intent encoder E_{int}:

$$v^* = E_{\text{int}}\big(h(p_0)\big), \tag{21}$$

so sentence relevance is measured against the malicious behavior to be elicited rather than against a single canonical answer. We treat E_{int} as fixed and do not update it during adversarial optimization.

d) Relevance Scoring: To obtain a trainable sentence-selection score, we feed the sentence embedding and the intent embedding into a lightweight scoring function $f_{\phi_{\text{sent}}}$, whose parameters are shared with the sentence-level branch of the generator:

$$r_j = f_{\phi_{\text{sent}}}(u_j, v^*) = \mathbf{w}_{\text{sent}}^{\top}\big[u_j;\, v^*;\, u_j \odot v^*\big] + b_{\text{sent}}, \quad j = 1, \ldots, K, \tag{22}$$

where $[\cdot\,;\,\cdot]$ denotes vector concatenation, $\odot$ denotes element-wise multiplication, and $\{\mathbf{w}_{\text{sent}}, b_{\text{sent}}, \phi_{\text{sent}}\} \subseteq \Theta_G$. A larger r_j indicates that sentence S_j is estimated to be more useful for preserving the adversarial intent.

e) Budgeted Selection: Given a sentence compression ratio $\lambda_{\text{sent}} \in (0,1]$, we define the selected sentence set as the solution to the constrained optimization problem

$$\mathcal{T} = \arg\max_{\mathcal{J} \subseteq [K]} \sum_{j \in \mathcal{J}} r_j \quad \text{s.t.} \quad \sum_{j \in \mathcal{J}} \text{len}(S_j) \leq \lambda_{\text{sent}}\, \text{len}\big(\tilde{p}_{\text{rec}}\big), \tag{23}$$

and construct the sentence-level compressed prompt by concatenating the selected sentences in their original order:

$$\tilde{p}_{\text{sent}} = \text{concat}\left(\{S_j \mid j \in \mathcal{T} \text{ in original order}\}\right). \tag{24}$$

If the constraint in (23) yields the empty set, we deterministically retain the highest-scoring single sentence. Hence $|\mathcal{T}^{(k)}| \geq 1$ always holds during optimization.

f) Critic-Guided Sentence Update: At iteration k, we optimize the sentence-compression branch according to the same anchor-calibrated critic principle used in the token branch. Because $\mathcal{T}^{(k)}$ is produced by a discrete constrained selection operator, we do not differentiate through the exact $\arg\max$; instead, we define the sentence-branch reward

$$\omega_{\text{sent}}^{(k)} = 2\Big((1-\eta^{(k)})\rho_{\text{sent}}^{(k)} + \eta^{(k)} a_{\text{sent}}^{(k)}\Big) - 1, \tag{25}$$

and optimize the surrogate objective

$$\mathcal{L}_{\text{sent}}^{(k)} = -\omega_{\text{sent}}^{(k)} \frac{1}{|\mathcal{T}^{(k)}|} \sum_{j \in \mathcal{T}^{(k)}} r_j^{(k)}, \tag{26}$$

where $a_{\text{sent}}^{(k)} \in \{0,1\}$ is the behavior-level anchor label of the sentence-compressed candidate, $\rho_{\text{sent}}^{(k)} \in (0,1)$ is the discriminator critic score, and $r_j^{(k)}$ is the trainable sentence score defined in (22). Because $|\mathcal{T}^{(k)}| \geq 1$, (26) is always well-defined. A higher reward strengthens the currently selected sentence subset, whereas a lower reward encourages the exploration of alternative compact sentence combinations.

3) Generator Optimization Objective: To optimize the generator, we combine the token-level loss $\mathcal{L}_{\text{token}}^{(k)}$ and the sentence-level loss $\mathcal{L}_{\text{sent}}^{(k)}$ into a unified objective at iteration k:

$$\mathcal{L}_{\text{total}}^{(k)} = \alpha_{\text{tok}}^{(k)} \mathcal{L}_{\text{token}}^{(k)} + \alpha_{\text{sent}}^{(k)} \mathcal{L}_{\text{sent}}^{(k)} + \lambda_{\text{reg}} \mathcal{R}(\Theta_G), \tag{27}$$

where Θ_G denotes the trainable parameter set of the generator, and λ_{reg} controls the impact of the regularization term $\mathcal{R}(\Theta_G)$. To modulate the trade-off between fine-grained token pruning and coarse-grained sentence pruning, we retain the dynamic weighting schedule

$$\alpha_{\text{tok}}^{(k)} = \frac{1}{1 + \exp(-\beta \cdot k)}, \qquad \alpha_{\text{sent}}^{(k)} = 1 - \alpha_{\text{tok}}^{(k)}, \tag{28}$$

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

where β governs the rate at which the contribution of each loss component evolves over iterations. Finally, the regularization term is defined as the ℓ_2 norm of the generator parameters:

$$\mathcal{R}(\Theta_G) = \|\Theta_G\|_2^2. \tag{29}$$

This objective keeps both compression branches inside a single two-agent optimization loop: the generator is optimized with dense critic guidance, while the behavior-level anchor keeps the optimization direction consistent with the final success definition.

B. Discriminator

a) Behavior-Level Anchor: Given the candidate prompt $\tilde{p}$, we first obtain the model response:

$$y = M_{\theta_M}(\tilde{p}), \quad y \in \mathcal{Y}, \tag{30}$$

where $\mathcal{Y}$ is the target LLM's output space. We then define the behavior-level anchor label

$$a(\tilde{p}, y; h(p_0)) = \mathbb{1}[y \text{ is non-refusal and } \operatorname{match}(y, h(p_0)) \geq \delta]. \tag{31}$$

During optimization, $a(\tilde{p}, y; h(p_0))$ is instantiated by a lightweight proxy aligned with the behavioral notion of success used in Section IV.

b) Online Critic: The discriminator maps a prompt–response pair to a scalar critic score

$$\rho(\tilde{p}, y) = D_{\Theta_D}(\tilde{p}, y) \in (0, 1), \tag{32}$$

which measures how compatible the current pair is with the target harmful behavior under the learned frontier of the search process. The anchor label in (31) remains the unique criterion used for replay labeling, success counting, early stopping, and final prompt selection.

c) Replay Memory: We maintain a replay memory

$$\mathcal{M} = \left\{ \left(\tilde{p}^{(m)}, y^{(m)}, a^{(m)} \right) \right\}_{m=1}^{N_{\mathcal{M}}}, \tag{33}$$

where $N_{\mathcal{M}} = |\mathcal{M}|$ is the current number of stored prompt–response pairs and

$$a^{(m)} = a\left(\tilde{p}^{(m)}, y^{(m)}; h(p_0) \right). \tag{34}$$

Since the optimization loop in Algorithm 1 is defined for a fixed source prompt p_0, all replayed pairs in $\mathcal{M}$ are anchored to the same harmful-intent descriptor $h(p_0)$. During critic optimization at iteration k, we sample an index mini-batch

$$\mathcal{U}^{(k)} \subseteq \{1, \ldots, N_{\mathcal{M}}\}, \qquad |\mathcal{U}^{(k)}| = U^{(k)}. \tag{35}$$

d) Joint Feature Extraction: For each $(\tilde{p}^{(m)}, y^{(m)})$, we encode their interaction via a joint encoder g_{ϕ_D}:

$$x^{(m)} = g_{\phi_D}(\tilde{p}^{(m)}, y^{(m)}) \in \mathbf{R}^d, \tag{36}$$

where g_{ϕ_D} employs self-attention layers to capture the semantic alignment between the compressed prompt and the corresponding response.

e) Critic Head: The joint feature is mapped to a scalar critic score through a logistic head:

$$\rho^{(m)} = D_{\Theta_D}(\tilde{p}^{(m)}, y^{(m)}) = \sigma(\mathbf{w}_{\text{cls}}^\top x^{(m)} + b_{\text{cls}}),$$
$$\sigma(x) = \frac{1}{1 + e^{-x}}, \tag{37}$$

where $\mathbf{w}_{\text{cls}} \in \mathbf{R}^d$ and $b_{\text{cls}} \in \mathbf{R}$ are learnable parameters.

f) Compactness-Aware Critic Loss: To make the critic useful for alternating optimization, we optimize it with both anchor supervision and frontier ranking. At iteration k, the anchor-classification term is

$$\mathcal{L}_{\text{BCE}}^{(k)} = -\frac{1}{U^{(k)}} \sum_{m \in \mathcal{U}^{(k)}} \left[a^{(m)} \log \rho^{(m)} + (1 - a^{(m)}) \log(1 - \rho^{(m)}) \right]. \tag{38}$$

In addition, we construct a hard-pair set on the current index mini-batch:

$$\mathcal{H}^{(k)} = \left\{ (u, v) \in \mathcal{U}^{(k)} \times \mathcal{U}^{(k)} \mid a^{(u)} > a^{(v)} \right\} \cup \left\{ (u, v) \in \mathcal{U}^{(k)} \times \mathcal{U}^{(k)} \mid a^{(u)} = a^{(v)} = 1, \ \operatorname{len}(\tilde{p}^{(u)}) < \operatorname{len}(\tilde{p}^{(v)}) \right\}, \tag{39}$$

which encourages the critic to score successful pairs above unsuccessful pairs and, among successful pairs, to prefer shorter prompts. The corresponding ranking term is

$$\mathcal{L}_{\text{rank}}^{(k)} = \begin{cases} \dfrac{1}{|\mathcal{H}^{(k)}|} \displaystyle\sum_{(u,v) \in \mathcal{H}^{(k)}} \max\left(0, \mu - \rho^{(u)} + \rho^{(v)}\right), & |\mathcal{H}^{(k)}| > 0, \\ 0, & |\mathcal{H}^{(k)}| = 0. \end{cases} \tag{40}$$

The full discriminator objective at iteration k is

$$\mathcal{L}_D^{(k)} = \mathcal{L}_{\text{BCE}}^{(k)} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}}^{(k)}. \tag{41}$$

g) Alternating Update Role: During adversarial training, the replay memory continuously absorbs new samples generated by the evolving generator, and the discriminator is repeatedly updated on this changing frontier. Consequently, the discriminator is not merely a static judge of a pre-existing label distribution; it is an online critic that tracks the generator's current search region and feeds back dense preference information for the next generator step, while remaining anchored to the same behavior-level success definition used throughout the method.

C. Adversarial Compression Loop

a) Initialization: At timestep $k = 0$, the generator starts from the reconstructed prompt $\tilde{p}_{\text{rec}}$ and initializes the token-level and sentence-level compression ratios $\lambda_{\text{tok}}^{(0)}$ and $\lambda_{\text{sent}}^{(0)}$. These initial budgets are chosen to be relatively loose so that the early search phase preserves semantic fidelity. In parallel, the discriminator

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

is initialized as an online critic, and the anchor-mixing range $[\eta_{\min}, \eta_{\max}]$ is specified in advance.

b) Candidate Prompt Generation: At each iteration k, the generator produces a set of candidate compressed prompts

$$\mathcal{X}^{(k)} = \left\{ \tilde{p}_{\text{tok}}^{(k)},\ \tilde{p}_{\text{sent}}^{(k)},\ \tilde{p}_{\cap}^{(k)} \right\} = \left\{ \tilde{p}_q^{(k)} \right\}_{q \in \mathcal{C}}, \tag{42}$$

where $\mathcal{C} = \{\text{tok}, \text{sent}, \cap\}$, $\tilde{p}_{\text{tok}}^{(k)}$ is the token-level compressed prompt generated by deterministic top-$\widetilde{N}^{(k)}$ selection, $\tilde{p}_{\text{sent}}^{(k)}$ is the sentence-level compressed prompt, and

$$\tilde{p}_{\cap}^{(k)} = \text{Merge}\left(\tilde{p}_{\text{tok}}^{(k)}, \tilde{p}_{\text{sent}}^{(k)} \right), \tag{43}$$

with $\text{Merge}(\cdot,\cdot)$ retaining the tokens that are selected by the token branch and also lie inside the selected sentence subset, while preserving their original order. The merged candidate is used to enrich the current search frontier and does not define an additional learnable generator branch.

c) Joint Evaluation: Each candidate $\tilde{p}_q^{(k)} \in \mathcal{X}^{(k)}$ is evaluated after querying the target LLM. The target LLM first produces a response

$$y_q^{(k)} = M_{\theta_M}\left(\tilde{p}_q^{(k)} \right), \tag{44}$$

and the corresponding behavior-level anchor label is

$$a_q^{(k)} = a\left(\tilde{p}_q^{(k)}, y_q^{(k)}; h(p_0) \right), \quad q \in \mathcal{C}. \tag{45}$$

In parallel, the discriminator outputs a critic score

$$\rho_q^{(k)} = D_{\Theta_D}\left(\tilde{p}_q^{(k)}, y_q^{(k)} \right). \tag{46}$$

For notational convenience, we define

$$a_{\text{tok}}^{(k)} := a_q^{(k)}\big|_{q=\text{tok}}, \qquad a_{\text{sent}}^{(k)} := a_q^{(k)}\big|_{q=\text{sent}}, \tag{47}$$

$$\rho_{\text{tok}}^{(k)} := \rho_q^{(k)}\big|_{q=\text{tok}}, \qquad \rho_{\text{sent}}^{(k)} := \rho_q^{(k)}\big|_{q=\text{sent}}. \tag{48}$$

All prompt–response pairs are inserted into the replay memory $\mathcal{M}$, and prompts with $a_q^{(k)} = 1$ are added to the successful-prompt set $\mathcal{P}_{\text{succ}}$.

d) Query Accounting Protocol: During the adversarial compression loop, each target-model call of the form

$$M_{\theta_M}\left(\tilde{p}_q^{(k)} \right)$$

counts as exactly one search query. Therefore, one full iteration contributes at most $|\mathcal{C}| = 3$ search queries. The query-efficiency metric reported in Section V counts only these optimization-stage target-model calls until the first successful prompt is found. By contrast, repeated response sampling used in the final ASR protocol is excluded from this statistic.

e) Alternating Two-Agent Update: At iteration k, we set the anchor-mixing coefficient by

$$\eta^{(k)} = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \frac{k-1}{\max(K_{\max} - 1, 1)}. \tag{49}$$

The generator is then updated through Eqs. (18) and (26), where the critic score provides dense frontier feedback and the anchor label keeps the update aligned with the final success definition. After the generator update, we sample an index mini-batch $\mathcal{U}^{(k)} \subseteq \{1, \ldots, N_{\mathcal{M}}\}$, construct the hard-pair set $\mathcal{H}^{(k)}$ using (39), and update the discriminator by minimizing $\mathcal{L}_D^{(k)}$. In this way, the generator and discriminator are optimized alternately on the same evolving search frontier.

f) Compression Schedule: To progressively tighten prompt length while preserving attack utility, the compression ratios are decayed over iterations:

$$\lambda_{\text{tok}}^{(k+1)} = \gamma_{\text{tok}} \lambda_{\text{tok}}^{(k)}, \quad \lambda_{\text{sent}}^{(k+1)} = \gamma_{\text{sent}} \lambda_{\text{sent}}^{(k)}, \quad \gamma_{\text{tok}}, \gamma_{\text{sent}} \in (0,1), \tag{50}$$

where γ_{tok} and γ_{sent} are the decay factors for token-level and sentence-level compression, respectively.

g) Adaptive Convergence Criteria: When $\mathcal{P}_{\text{succ}} \neq \emptyset$, we define the current best successful length as

$$\ell_{\min}^{(k)} = \min_{\tilde{p} \in \mathcal{P}_{\text{succ}}} \text{len}(\tilde{p}). \tag{51}$$

The adversarial loop terminates when two conditions are jointly satisfied: the shortest successful prompt no longer becomes shorter, and the variance of the total generator loss $\mathcal{L}_{\text{total}}$ over a patience window P falls below ϵ. The maximum iteration count $K_{\max}$ is retained as a safety upper bound.

h) Final Prompt Selection: Upon termination, the final adversarial prompt is selected as

$$\tilde{p}^* = \begin{cases} \arg\min_{\tilde{p} \in \mathcal{P}_{\text{succ}}} \text{len}(\tilde{p}), & \mathcal{P}_{\text{succ}} \neq \emptyset, \\ \tilde{p}_{\text{rec}}, & \mathcal{P}_{\text{succ}} = \emptyset. \end{cases} \tag{52}$$

Because the same behavior-level anchor governs both optimization-time success recording and final output selection, the search mechanism and the terminal decision rule remain fully consistent.

IV. Main Experimental Analysis

A. Experimental Setup

We evaluate ComJail on two public jailbreak benchmarks using Attack Success Rate (ASR) as the primary metric, compare it against representative white-box and black-box baselines, and report both automatic and human evaluations.

1) Datasets:

a) AdvBench: AdvBench [19] is a widely used benchmark for evaluating jailbreak attacks on aligned LLMs. It contains 520 manually verified harmful instructions spanning diverse categories such as hate speech, violence, illegal activities, harassment, and misinformation. Following prior work [35], [36], [37], we use the standard 100-sample subset (*AdvBench-Sub*) in our main experiments to enable direct comparison with existing methods. We additionally evaluate on the full 520-sample set (*AdvBench*) to assess robustness over a broader range of harmful intents.

b) JailbreakBench: JailbreakBench [38] complements AdvBench by focusing on policy-relevant jailbreak behaviors observed in practical moderation settings. It includes more challenging cases such as role-play attacks, indirect intent expression, context stacking, and multi-turn or ambiguously framed queries. We use this benchmark to test whether ComJail generalizes beyond standard harmful instructions and remains effective under more realistic adversarial conditions.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

2) Evaluation Metric: We use Attack Success Rate (ASR) as the primary metric:

$$\text{ASR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[f(x_i) = \text{success}\right], \tag{53}$$

where $f(x_i)$ denotes whether prompt x_i satisfies the attack objective at the prompt level. During optimization, a single response is preliminarily scored by a lightweight behavior-matching proxy aligned with the final success criterion; an auxiliary LLM judge is invoked only for ambiguous cases.

To reduce variance from stochastic decoding, we generate k responses for each prompt using independent inference restarts. A prompt is counted as successful only if at least half of the generated responses are successful. Formally, let $s_{i,j} \in \{0, 1\}$ denote the success indicator of the j-th response for prompt x_i, where $j = 1, \ldots, k$. Then, the prompt-level success function is defined as

$$f(x_i) = \begin{cases} \text{success}, & \text{if } \sum_{j=1}^{k} s_{i,j} \geq \left\lceil \frac{k}{2} \right\rceil, \\ \text{failure}, & \text{otherwise.} \end{cases} \tag{54}$$

In all main experiments, we set $k = 5$, so a prompt is regarded as successful only when at least three out of the five generated responses satisfy the attack objective. All methods are evaluated under the same decoding settings, model versions, and safety configurations. Unless otherwise stated, ASR is reported as a percentage, and confidence intervals are estimated by bootstrapping over prompts.

3) Evaluation Settings: We report both GPT Evaluation and Human Evaluation.

a) GPT Evaluation: For scalable and reproducible evaluation, we use GPT-4o as an automatic judge to determine whether a model response fulfills the harmful instruction. Specifically, GPT-4o is given the original harmful query and each corresponding model output, and returns a binary "Yes" or "No" with a brief rationale. For each prompt, the final judgment follows the same prompt-level criterion as above: a prompt is counted as successful only if at least three out of the five generated responses are judged as successful. The final ASR is then computed over prompts.

b) Human Evaluation: To validate the automatic judgments, we additionally conduct human evaluation on a randomly selected subset of samples. Each response is independently and blindly annotated by at least five researchers with expertise in LLM jailbreak. A response is labeled as successful if it satisfies the harmful instruction without refusal, evasion, or safety disclaimers. For each prompt, final labels are determined by majority voting across annotators at the response level and by the same prompt-level criterion across the five generated responses.

4) Baselines: We compare ComJail with representative jailbreak baselines in both white-box and black-box settings. All baselines are implemented using the EasyJailbreak framework [39].

a) White-Box Attacks: We consider Greedy Coordinate Gradient (GCG) [19], which optimizes adversarial suffix tokens using gradients, and AutoDAN [18], which adopts a hierarchical genetic algorithm to evolve fluent yet effective jailbreak prompts.

b) Black-Box Attacks: We include In-Context Attack (ICA) [20], PAIR [23], DeepInception [21], ReNeLLM [26], and DrAttack [36]. These methods cover representative black-box strategies based on in-context demonstrations, iterative prompt refinement, nested or role-play prompts, and multi-turn prompt decomposition.

5) Models: We evaluate ComJail on both proprietary and open-source LLMs. Proprietary targets include GPT-3.5-Turbo and GPT-4 [40], Claude-2 [41], and Gemini-Pro [42]. Open-source targets include DeepSeek-R1 [43], DeepSeek-V3 [44], Vicuna-7B/13B [45], and LLaMA2-7B/13B [46]. Unless otherwise specified, all models are queried through their official APIs or inference platforms under default safety settings.

6) Implementation Details: All experiments are conducted on a high-performance cluster equipped with NVIDIA A100 80 GB GPUs, AMD EPYC CPUs, and 256 GB RAM. We implement ComJail in Python 3.10 with PyTorch 2.4.0, CUDA 12.1, and cuDNN, and use Hugging Face Transformers together with Accelerate for model loading and inference management. Half-precision inference is adopted when supported to improve efficiency. In the adaptive optimization loop, we set the patience window to $P = 5$ and the loss variance threshold to $\epsilon = 1 \times 10^{-3}$ to avoid premature stopping while limiting unnecessary iterations.

B. Main Results

1) Performance on AdvBench-Sub: Table II reports the ASR of ComJail and the baselines under both GPT-4o and human evaluation.

a) GPT Evaluation: ComJail achieves strong ASR across both proprietary and open-source models. On proprietary models, it reaches 86% on GPT-3.5-Turbo, 74% on GPT-4, 42% on Claude-2, and 88% on Gemini-Pro, exceeding the best-performing baseline, DrAttack, by 8%, 11%, 15%, and 9%, respectively.

On open-source models, it reaches 84% on DeepSeek-V3, 67% on DeepSeek-R1, 91% on Vicuna-7B, 90% on Vicuna-13B, 58% on LLaMA2-7B, and 71% on LLaMA2-13B. The gains remain clear on the two recent DeepSeek models, where ComJail outperforms DrAttack by 8% on DeepSeek-V3 and 18% on DeepSeek-R1, indicating that the method maintains strong attack capability on newer open-source targets. Compared with AutoDAN, the strongest white-box baseline on several open-source settings, ComJail improves by 3% on Vicuna-7B, 14% on Vicuna-13B, and 69% on LLaMA2-13B, although AutoDAN remains higher on LLaMA2-7B. ComJail also substantially outperforms black-box baselines such as ICA, PAIR, and DeepInception. For example, DeepInception obtains only 27% on Gemini-Pro and 1% on GPT-4, whereas ComJail achieves 88% and 74%, respectively.

b) Human Evaluation: Human evaluation shows the same overall trend, with ComJail achieving the best ASR on all

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON OF JAILBREAK ATTACKS ON ADVBENCH-SUB, EVALUATED BY GPT AND HUMAN EVALUATION

Attack Type	Attack Methods	Proprietary Models				Open-Source Models					
		GPT-3.5-Turbo	GPT-4	Claude-2	Gemini-Pro	DeepSeek-V3	DeepSeek-R1	Vicuna-7B	Vicuna-13B	LLaMA2-7B	LLaMA2-13B
GPT Evaluation											
White-Box	GCG [19]	6%	0%	1%	1%	1%	0%	88%	86%	46%	38%
	AutoDAN [18]	39%	3%	10%	64%	56%	18%	88%	76%	64%	2%
Black-Box	ICA [20]	1%	0%	0%	0%	0%	0%	49%	81%	1%	0%
	PAIR [23]	12%	10%	1%	12%	14%	6%	76%	70%	3%	4%
	DeepInception [21]	0%	1%	5%	27%	22%	8%	29%	7%	6%	8%
	ReNeLLM [26]	48%	13%	18%	48%	46%	31%	54%	47%	30%	44%
	DrAttack [36]	78%	63%	27%	79%	76%	49%	82%	63%	50%	62%
	ComJail (Ours)	86%	74%	42%	88%	84%	67%	91%	90%	58%	71%
Human Evaluation											
White-Box	GCG [19]	9%	0%	0%	6%	2%	0%	96%	95%	41%	40%
	AutoDAN [18]	29%	0%	0%	52%	48%	14%	93%	79%	12%	0%
Black-Box	ICA [20]	0%	1%	0%	0%	0%	0%	45%	84%	0%	0%
	PAIR [23]	12%	6%	0%	28%	19%	8%	74%	70%	1%	2%
	DeepInception [21]	5%	4%	3%	13%	11%	5%	20%	13%	0%	0%
	ReNeLLM [26]	49%	15%	11%	50%	47%	33%	53%	49%	27%	42%
	DrAttack [36]	84%	80%	26%	80%	79%	58%	98%	80%	37%	44%
	ComJail (Ours)	**91%**	**89%**	**40%**	**89%**	**88%**	**74%**	**99%**	**96%**	**50%**	**59%**

TABLE III
PERFORMANCE COMPARISON OF JAILBREAK ATTACKS ON ADVBENCH, EVALUATED BY HUMAN EVALUATION

Attack Methods	GPT-3.5-Turbo	GPT-4	LLaMA2-7B	Vicuna-7B
GCG [19]	8.4%	1.2%	57.0%	88.0%
AutoDAN [18]	64.5%	12.4%	21.6%	83.5%
DeepInception [21]	35.6%	14.6%	18.5%	41.2%
ICA [20]	8.1%	4.8%	7.6%	54.1%
ComJail (Ours)	**87.0%**	**94.2%**	**65.4%**	**93.3%**

TABLE IV
PERFORMANCE COMPARISON OF JAILBREAK ATTACKS ON JAILBREAKBENCH, EVALUATED BY HUMAN EVALUATION

Attack Methods	GPT-3.5-Turbo	GPT-4	Vicuna-13B	LLaMA2-7B
GCG [19]	6.0%	2.0%	95.0%	8.0%
PAIR [23]	74.0%	52.0%	79.0%	4.0%
DrAttack [36]	56.0%	75.0%	96.0%	53.0%
ComJail (Ours)	**76.0%**	**79.0%**	**98.0%**	**61.0%**

tested models. On proprietary models, it attains 91% on GPT-3.5-Turbo, 89% on GPT-4, 40% on Claude-2, and 89% on Gemini-Pro, outperforming DrAttack by 7%, 9%, 14%, and 9%, respectively.

On open-source models, it reaches 88% on DeepSeek-V3, 74% on DeepSeek-R1, 99% on Vicuna-7B, 96% on Vicuna-13B, 50% on LLaMA2-7B, and 59% on LLaMA2-13B. The advantage on DeepSeek-V3 and DeepSeek-R1 remains substantial under human judgment, with gains of 9% and 16% over DrAttack, respectively, which is consistent with the GPT-based evaluation. The improvement over AutoDAN is especially clear on the LLaMA2 models, where AutoDAN achieves only 12% on LLaMA2-7B and 0% on LLaMA2-13B. Against black-box baselines on open-source targets, ComJail also remains superior, with 99% on Vicuna-7B and 96% on Vicuna-13B, clearly exceeding alternatives such as PAIR and ReNeLLM. Overall, these results indicate that ComJail produces more effective adversarial prompts under both GPT-based evaluation and human evaluation.

2) Performance on AdvBench: Table III reports the human-evaluated ASR of ComJail and the baselines on the full AdvBench dataset. ComJail achieves the best performance across both proprietary and open-source models.

a) Proprietary Models: On proprietary models, ComJail reaches 87.0% on GPT-3.5-Turbo and 94.2% on GPT-4. On GPT-3.5-Turbo, it outperforms AutoDAN by 22.5%. On GPT-4, it substantially exceeds DeepInception and AutoDAN, which achieve 14.6% and 12.4%, respectively. These results indicate that ComJail remains effective under restricted-access settings.

b) Open-Source Models: ComJail also performs strongly on open-source models, achieving 65.4% on LLaMA2-7B and 93.3% on Vicuna-7B. This improves over GCG by 8.4% on LLaMA2-7B and 5.3% on Vicuna-7B. The results further support the effectiveness of the proposed compression-aware strategy across different open-source architectures.

3) Performance on JailbreakBench: Table IV reports the human-evaluated ASR on JailbreakBench. ComJail consistently achieves the best overall performance, showing cross-dataset generalization.

a) Proprietary Models: On proprietary models, ComJail obtains 76.0% on GPT-3.5-Turbo and 79.0% on GPT-4. These results exceed the best-performing baselines by 2.0% and 4.0%, respectively, compared with PAIR on GPT-3.5-Turbo and DrAttack on GPT-4. This indicates that ComJail transfers effectively to restricted-access targets.

b) Open-Source Models: On open-source models, ComJail achieves 98.0% on Vicuna-13B and 61.0% on LLaMA2-7B. The gain over DrAttack is 2.0% on Vicuna-13B and 8.0% on LLaMA2-7B, while all other baselines remain lower. These results further demonstrate the robustness of ComJail across different model families.

V. FURTHER ANALYSIS

A. ASR Vs. Average Number of Search Queries

We further examine the practical efficiency of ComJail by jointly considering attack success and online search cost. In black-box jailbreaking, these two dimensions are equally important: ASR reflects whether an attack is effective, while the number of search queries determines whether the attack remains

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

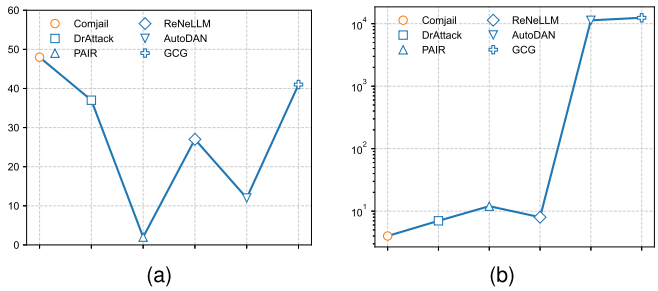


Fig. 3. Comparison of ASR and search-stage query efficiency across attack methods. (a) ASR of different attack methods. (b) Average number of search queries required per attack (log scale). ComJail achieves the strongest overall trade-off, combining the highest attack success with the lowest search cost among the black-box baselines.

practical under API cost, latency, and rate-limit constraints. A strong black-box attacker should therefore not only achieve a high success rate, but do so with a small search budget.

Following the query accounting protocol in Section III-C, one *search query* denotes one target-model call issued during prompt search. For ComJail, each optimization iteration evaluates at most three candidates, namely the token-level, sentence-level, and merged prompts, so one iteration contributes at most three search queries. We report the average number of search queries required to obtain the first successful prompt. Repeated response sampling for final ASR evaluation is excluded from this metric.

1) Attack Effectiveness Analysis: As shown in Fig. 3(a), ComJail occupies a favorable region of the ASR–search-cost plane. On the LLaMA2-7B target, ComJail achieves a 50% ASR under the human evaluation protocol in Table II, outperforming the strongest black-box baseline DrAttack as well as other competitive baselines such as ReNeLLM. Notably, this advantage is obtained without sacrificing search efficiency, indicating that the gain of ComJail does not come from a heavier search process, but from producing more attack-effective candidates under the same black-box interaction setting.

Compared with methods that rely on rigid prompt structures or longer optimization trajectories, ComJail benefits from compression-aware adversarial generation, which preserves the most attack-relevant content while removing redundant prompt structure. As a result, the generated prompts remain strong enough to induce harmful responses, yet compact enough to maintain a clear efficiency advantage during search. This combination makes ComJail particularly competitive in realistic black-box settings, where both attack success and online interaction cost matter.

2) Query Efficiency Analysis: Fig. 3(b) shows that ComJail also achieves the lowest average search cost among the compared black-box methods. The gap is substantial: ComJail requires fewer search queries to reach a successful prompt than DrAttack, PAIR, and ReNeLLM, while simultaneously maintaining the best ASR in Fig. 3(a). This means that ComJail improves not only the final attack outcome, but also the efficiency of the search process itself.

The result is especially important from a deployment perspective. In practical black-box attacks, each additional query increases monetary cost, latency, and the chance of triggering detection or rate-limiting mechanisms. By reducing the number of search-stage target-model calls while maintaining stronger attack success, ComJail offers a more favorable effectiveness–cost trade-off than existing black-box baselines. In other words, ComJail is not merely a higher-ASR attacker; it is a more search-efficient attacker as well.

For completeness, Fig. 3(b) also includes white-box baselines. These methods remain useful reference points because they highlight the efficiency gap between direct token-level optimization and compression-aware black-box search. Even under this broader comparison, ComJail remains highly competitive: it surpasses the white-box references in ASR on this target while using dramatically fewer search queries than optimization-heavy baselines such as GCG and AutoDAN. This further supports the core claim of our framework that compactness-aware search can improve both effectiveness and efficiency, rather than trading one for the other.

TABLE V
ABLATION STUDY OF INDIVIDUAL COMPONENTS WITHIN THE PROPOSED FRAMEWORK. PERFORMANCE DEGRADATION IS EVALUATED RELATIVE TO THE FULL CONFIGURATION, WITH THE ABSOLUTE DECREASE IN ASR PRESENTED IN PARENTHESES.

Attack Methods	GPT-4	Vicuna-7B
ComJail (Full)	**89%**	**99%**
w/o SCR	77% (-12%)	89% (-10%)
w/o TLC	73% (-16%)	85% (-14%)
w/o SLC	80% (-9%)	91% (-8%)
w/o SCR & TLC	65% (-24%)	74% (-25%)
w/o SCR & SLC	69% (-20%)	78% (-21%)
w/o TLC & SLC	63% (-26%)	70% (-29%)

B. Ablation Study

We conduct an ablation study to evaluate the contribution of individual components within the proposed framework. The evaluated modules include Similarity-Based Contextual Reconstruction (SCR), Token-Level Compression (TLC), and Sentence-Level Compression (SLC).

1) Experimental Setup: We perform the ablation experiments on two representative models, including the proprietary GPT-4 and the open-source Vicuna-7B. For each configuration, we systematically remove one or multiple components and measure the resulting ASR under the same evaluation protocol as in the main experiments. Table V summarizes the results, where the value in parentheses denotes the absolute decrease in ASR relative to the full configuration.

2) Results and Analysis: As shown in Table V, removing any component leads to a clear drop in ASR on both target models, indicating that all three modules contribute to the effectiveness of ComJail. Among single-component removals, omitting TLC causes the largest degradation, with ASR dropping by 16% on GPT-4 and 14% on Vicuna-7B, suggesting that token-level selection plays the most critical role in preserving adversarial

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ATTACK SUCCESS RATE (%) UNDER FIXED TOKEN BUDGETS ON ADVBENCH-SUB. THE EVALUATION COMPARES THE PROPOSED METHOD AGAINST BASELINE TRUNCATION AND COMPRESSION STRATEGIES UNDER MATCHED LENGTH CONSTRAINTS AND IDENTICAL INFERENCE CONFIGURATIONS.

Model	Method	96 Tok	128 Tok	160 Tok	192 Tok
GPT-4 [40]	Random Truncation	18%	27%	35%	42%
	Head Truncation	21%	31%	39%	46%
	Tail Truncation	23%	33%	42%	49%
	LLMLingua-2	42%	55%	66%	73%
	ComJail	**52%**	**64%**	**72%**	**78%**
Vicuna-7B [45]	Random Truncation	24%	35%	46%	54%
	Head Truncation	28%	39%	50%	58%
	Tail Truncation	31%	42%	53%	61%
	LLMLingua-2	54%	70%	79%	85%
	ComJail	**75%**	**86%**	**91%**	**94%**

utility under compression. Removing SCR also yields a notable decline, showing that contextual reconstruction provides a more coherent semantic scaffold for the subsequent compression stages, while removing SLC results in a smaller but still consistent reduction.

The degradation becomes larger when two components are removed simultaneously. In particular, the *w/o TLC & SLC* and *w/o SCR & TLC* settings produce the largest ASR drops, confirming that the three modules are complementary rather than redundant. Overall, SCR provides a more coherent reconstruction scaffold, whereas TLC and SLC improve information density and preserve adversarially useful content under limited prompt budgets.

C. Evaluation Under Fixed Token Budgets

We conduct a controlled evaluation under fixed token budgets to more directly examine the role of the proposed compression mechanism. The original ablation study mainly verifies the necessity of individual components within the framework. By contrast, the present experiment isolates whether the gain of ComJail can still be observed when the final prompt length is explicitly constrained. In this way, the comparison focuses on compression quality rather than on unconstrained prompt generation.

1) Experimental Setup: We perform the controlled evaluation on two representative target models, including the proprietary GPT-4 and the open-source Vicuna-7B. We set the token budgets to 96, 128, 160, and 192 tokens, which cover highly constrained and moderately relaxed prompt regimes while remaining compatible with the evaluation template used in our experiments. For each target model, the proposed method is compared with four baseline strategies, including Random Truncation, Head Truncation, Tail Truncation, and LLMLingua-2. For fairness, all compared methods are evaluated under the same prompt construction setting, the same final token budget, and identical inference configurations. In particular, the reported budget always refers to the final adversarial prompt budget after compression or truncation, so the comparison is made under strictly matched length constraints. The ASR is reported based on human evaluation. Table VI summarizes the corresponding results.

2) Results and Analysis: As shown in Table VI, the proposed method consistently maintains a higher ASR across all evaluated token budgets on both target models. This result suggests that the observed advantage is not merely a byproduct of making prompts shorter. Rather, it is closely related to the compression-aware optimization strategy, which better preserves adversarially useful content under strict length constraints.

Baseline methods exhibit a clear performance drop when the budget is tight. On GPT-4, Random Truncation, Head Truncation, Tail Truncation, and LLMLingua-2 achieve 18%, 21%, 23%, and 42% ASR, respectively, under the 96-token budget. On Vicuna-7B, the corresponding ASR values are 24%, 28%, 31%, and 54%. Under the same budget, the proposed method reaches 52% on GPT-4 and 75% on Vicuna-7B. These results indicate that, when the available token budget is severely restricted, direct truncation or static compression tends to damage prompt continuity and remove adversarially relevant information. In contrast, the joint optimization procedure better preserves compact but attack-effective prompt structures.

The advantage of ComJail remains stable as the token budget increases. On GPT-4, it exceeds LLMLingua-2 by 10%, 9%, 6%, and 5% under 96, 128, 160, and 192 token budgets, respectively. On Vicuna-7B, the corresponding margins are 21%, 16%, 12%, and 9%. Although all methods benefit from larger budgets, the relative ranking remains unchanged across all evaluated settings. This consistency indicates that the gain is systematic rather than tied to a single budget value. Overall, the controlled fixed-budget evaluation complements the original ablation study by showing that the effectiveness of ComJail is associated with how adversarial utility is preserved during compression, instead of with prompt shortening alone.

D. Evaluation Under Additional Defense Mechanisms

We further evaluate ComJail under stronger defense-aware settings. Since practical LLM services often use layered screening pipelines, ASR measured only under default safety settings or lightweight filters may overestimate practical performance under layered screening pipelines. We therefore evaluate ComJail under four representative defenses: Llama Guard-2-8B, perplexity-based filtering, SmoothLLM, and a system-prompt-based Semantic Firewall.

1) Experimental Setup: We use ASR as the primary metric under defense-aware settings. An attack is counted as successful only if the adversarial prompt survives the corresponding defense and then elicits harmful output from the target model. Thus, the reported ASR reflects both prompt survival under defense and downstream attack effectiveness.

We evaluate three methods on GPT-4 and Vicuna-7B: DrAttack, AutoDAN, and ComJail. Llama Guard-2-8B [47] is used as an input-side safety classifier, and prompts classified as unsafe are rejected before reaching the target model. For perplexity-based filtering [48], we compute prompt-level perplexity using a fixed estimator and calibrate the rejection threshold on a benign split. The threshold is set to the 95th percentile of the benign perplexity distribution and kept fixed across methods.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
ATTACK SUCCESS RATE (%) UNDER INDIVIDUAL DEFENSE MECHANISMS. LG-2 DENOTES LLAMA GUARD-2-8B, AND SEM.-FW DENOTES THE SYSTEM-PROMPT-BASED SEMANTIC FIREWALL. AN ATTACK IS COUNTED AS SUCCESSFUL ONLY WHEN THE PROMPT SURVIVES THE CORRESPONDING DEFENSE AND ELICITS HARMFUL OUTPUT FROM THE TARGET MODEL.

Model	Method	No Defense	+ LG-2	+ PPL	+ SmoothLLM	+ Sem.-FW
GPT-4 [40]	DrAttack [36]	80%	60%	56%	45%	41%
	AutoDAN [18]	0%	0%	0%	0%	0%
	ComJail	**89%**	**76%**	**73%**	**65%**	**61%**
Vicuna-7B [45]	DrAttack [36]	98%	87%	83%	69%	65%
	AutoDAN [18]	93%	79%	74%	60%	56%
	ComJail	**99%**	**95%**	**93%**	**85%**	**82%**

TABLE VIII
PROMPT PASS / SURVIVAL RATE (%) UNDER INDIVIDUAL DEFENSE MODULES. THE RESULTS QUANTIFY HOW OFTEN THE GENERATED PROMPTS SURVIVE EACH DEFENSE BEFORE FINAL DELIVERY TO THE TARGET MODEL. FOR SMOOTHLLM, THE REPORTED VALUE CORRESPONDS TO THE FRACTION OF PROMPTS THAT SURVIVE THE SMOOTHED DECISION RULE.

Model	Method	Pass@LG-2	Pass@PPL	Pass@SmoothLLM	Pass@Sem.-FW
GPT-4 [40]	DrAttack [36]	69%	64%	55%	50%
	AutoDAN [18]	61%	57%	48%	44%
	ComJail	**84%**	**81%**	**75%**	**71%**
Vicuna-7B [45]	DrAttack [36]	89%	85%	73%	69%
	AutoDAN [18]	80%	76%	66%	61%
	ComJail	**96%**	**94%**	**87%**	**84%**

We also evaluate a SmoothLLM-style wrapper [49] as a perturbation-aware defense. For each input, we generate 10 perturbed prompt variants and aggregate their safety outcomes by majority voting. If the aggregated decision indicates unsafe or unstable behavior, the original prompt is blocked before final delivery.

In addition, we include a system-prompt-based Semantic Firewall [50] as a semantic-aware defense. A frozen guard model is placed before the target model with a locked system prompt that rewrites the input into a canonical intent statement, infers the latent objective behind compressed or obfuscated phrasing, and classifies the restored intent under a fixed safety policy. Prompts predicted as unsafe are rejected before reaching the target model.

For fairness, all thresholds are calibrated once on the same benign validation split and then kept fixed across attack methods. Table VII reports ASR under different defense settings, and Table VIII reports the prompt pass/survival rate under the four individual defense modules.

2) Results and Analysis: Table VII shows that all methods degrade after stronger defenses are introduced. The reduction is more pronounced under SmoothLLM and the Semantic Firewall than under Llama Guard-2-8B and the PPL filter, especially on GPT-4. This suggests that perturbation-aware and semantic-aware defenses are more effective at reducing the attack margin of compact jailbreak prompts.

ComJail nevertheless remains the best-performing method under all evaluated defenses. On GPT-4, its ASR drops from 89% without defense to 76% with Llama Guard-2-8B, 73% with the PPL filter, 65% with SmoothLLM, and 61% with the Semantic Firewall. The corresponding values for DrAttack are 60%, 56%, 45%, and 41%, while AutoDAN remains at 0% throughout. On Vicuna-7B, ComJail reaches 95%, 93%, 85%, and 82% under the four defenses, still exceeding DrAttack at 87%, 83%, 69%, and 65%, and AutoDAN at 79%, 74%, 60%, and 56%. These results show that stronger deployment-time defenses reduce absolute ASR, but ComJail retains a clear relative advantage.

Table VIII reports the detector-side results. ComJail achieves the highest prompt survival rate under all four individual defenses on both target models, which is consistent with its stronger downstream ASR. The pass-rate gap is smaller under SmoothLLM and the Semantic Firewall than under Llama Guard-2-8B and the PPL filter, indicating that stronger defenses are more effective at reducing the defense-survival advantage of compressed jailbreak prompts.

E. Source-to-Target Transfer Evaluation

The original experiments show that ComJail is effective on multiple target models. However, model-wise ASR reported independently does not by itself establish transferability in the adversarial sense. A stricter protocol requires prompts to be optimized on one source model and then directly reused on unseen target models without any target-side modification. We therefore conduct source-to-target transfer experiments on a transfer-specific subset of AdvBench and include baselines for comparison.

1) Experimental Setup: We select GPT-3.5-Turbo, Vicuna-7B, and LLaMA2-7B as source models. For each source model, adversarial prompts are generated using DrAttack, AutoDAN, and ComJail. For each harmful behavior, the prompt optimized on the source model is then frozen and directly transferred to a diverse set of target models, including GPT-4, Claude-2, Gemini-Pro, Vicuna-13B, and LLaMA2-13B. No target-side adaptation, re-ranking, or additional search is performed during transfer. All methods are evaluated on the same transfer subset. For consistency, the self-model ASR in Table X is also computed on this identical subset. The detailed transfer ASR is reported in Table IX, and Table X compares the ASR on the source model itself with the mean ASR over all transferred target models to quantify performance retention after transfer.

2) Results and Analysis: As shown in Table IX, ComJail maintains strong transfer ASR across the evaluated source and target models. Its transfer performance is particularly strong within the same model family. For example, prompts optimized on Vicuna-7B achieve 90% ASR when transferred to Vicuna-13B. At the same time, cross-family transfer is also viable: prompts optimized on GPT-3.5-Turbo achieve 78% ASR on GPT-4 and 75% on Gemini-Pro.

Table X further shows that ComJail consistently retains a larger portion of its source-model effectiveness after direct transfer than the compared baselines. Using GPT-3.5-Turbo as the source model, ComJail attains 91% self-model ASR and 67.4% mean cross-model ASR, retaining about 74.1% of its source-model performance after transfer. The corresponding figures for DrAttack are 84% and 39.4%. Using Vicuna-7B as the source model, ComJail achieves 99% self-model ASR and 64.8% mean cross-model ASR, corresponding to a retention ratio of about 65.5%. The corresponding metrics for AutoDAN are 93% and 47.8%. Using LLaMA2-7B as the source model, ComJail obtains 50% self-model ASR and 39.0% mean cross-model ASR, retaining about 78.0% of its source-model

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
SOURCE-TO-TARGET TRANSFER ATTACK SUCCESS RATE (%). ADVERSARIAL PROMPTS ARE OPTIMIZED ON THE SELECTED SOURCE MODELS AND DIRECTLY EVALUATED ON UNSEEN TARGET MODELS WITHOUT MODIFICATION.

Source Model	Method	GPT-4	Claude-2	Gemini-Pro	Vicuna-13B	LLaMA2-13B	Mean Transfer ASR
GPT-3.5-Turbo [40]	DrAttack [36]	48%	22%	41%	54%	32%	39.4%
	AutoDAN [18]	21%	9%	17%	27%	17%	18.2%
	ComJail	**78%**	**44%**	**75%**	**81%**	**59%**	**67.4%**
Vicuna-7B [45]	DrAttack [36]	42%	18%	37%	61%	45%	40.6%
	AutoDAN [18]	50%	24%	44%	69%	52%	47.8%
	ComJail	**63%**	**39%**	**67%**	**90%**	**65%**	**64.8%**
LLaMA2-7B [46]	DrAttack [36]	21%	9%	18%	35%	29%	22.4%
	AutoDAN [18]	5%	2%	4%	8%	5%	4.8%
	ComJail	**38%**	**22%**	**41%**	**46%**	**48%**	**39.0%**

TABLE X
SELF-MODEL AND CROSS-MODEL TRANSFER PERFORMANCE (%). THE GAP METRIC INDICATES THE PERFORMANCE REDUCTION AFTER TRANSFERRING ADVERSARIAL PROMPTS FROM THE SOURCE MODEL TO UNSEEN TARGET MODELS.

Source Model	Method	Self-model ASR	Mean Cross-model ASR	Gap
GPT-3.5-Turbo [40]	DrAttack [36]	84%	39.4%	44.6%
	AutoDAN [18]	29%	18.2%	10.8%
	ComJail	**91%**	**67.4%**	**23.6%**
Vicuna-7B [45]	DrAttack [36]	98%	40.6%	57.4%
	AutoDAN [18]	93%	47.8%	45.2%
	ComJail	**99%**	**64.8%**	**34.2%**
LLaMA2-7B [46]	DrAttack [36]	37%	22.4%	14.6%
	AutoDAN [18]	12%	4.8%	7.2%
	ComJail	**50%**	**39.0%**	**11.0%**

performance. These results indicate that the compressed prompts produced by ComJail preserve transferable adversarial structure more effectively than the compared baselines, thereby providing stronger evidence for cross-model transferability than separate per-model ASR alone.

F. Training Stability Analysis

Adversarial generative frameworks involving min-max optimization can exhibit unstable training behavior, including oscillatory updates and collapse to low-utility solutions. To improve empirical stability, the proposed framework incorporates three design choices. First, a curriculum-based compression schedule is adopted. This schedule gradually tightens length constraints, allowing the generator to learn semantic reconstruction before aggressive pruning. Second, discrete budget constraints are imposed. These constraints restrict the search space and reduce the tendency of the generator to produce degenerate sequences in the early stage of training. Third, a convergence-aware termination mechanism is introduced. This mechanism stops optimization when length reduction saturates and the training loss becomes stable. It helps avoid unnecessary updates after convergence and reduces the risk of drifting toward sub-optimal solutions.

1) Quantitative Stability Metrics: To assess empirical stability, we tracked the training process across 5 independent runs with different random seeds for four representative target models. We report three descriptive metrics in Table XI. The Average Convergence Epochs metric denotes the average number of epochs required to satisfy the convergence-aware stopping criterion. The Final Loss Variance metric measures the variance of the generator total loss over the final 10 epochs before termination. The ASR Standard Deviation metric represents the standard deviation of the final ASR across the 5 runs.

TABLE XI
TRAINING STABILITY STATISTICS OVER INDEPENDENT RUNS. THE METRICS REPORT THE AVERAGE CONVERGENCE EPOCHS, FINAL LOSS VARIANCE, AND ASR STANDARD DEVIATION ACROSS THE EVALUATED TARGET MODELS.

Target Model	Avg. Conv. Epochs	Final Loss Var (10^{-3})	ASR Std. Dev.
Vicuna-7B [45]	17.8	1.12	2.3%
LLaMa2-7B [46]	22.1	1.45	2.9%
GPT-3.5-Turbo [40]	24.6	1.87	3.5%
GPT-4 [40]	29.3	2.41	4.2%

As shown in Table XI, the framework exhibits consistent convergence behavior across the evaluated architectures, although convergence speed and variance differ between open-source and black-box settings. For open-source models such as Vicuna-7B, the framework converges in 17.8 epochs with an ASR deviation of 2.3%. For black-box models such as GPT-4, convergence requires 29.3 epochs, which is likely related to the absence of gradient access and the stochasticity of application programming interface responses. Even in this setting, the final loss variance remains bounded and the ASR standard deviation is 4.2%, indicating that the training outcomes are relatively consistent across random seeds in the reported runs.

2) Convergence Behavior: The discriminator feedback loop provides an auxiliary signal that helps regularize the optimization process. In addition, the token-level weighting mechanism enables fine-grained updates during prompt refinement. Black-box optimization still introduces observable fluctuations in the loss curve. Nevertheless, across runs, the overall trend typically shows a rapid initial decrease followed by a gradual plateau. This pattern suggests that the optimization approaches a stable operating region under the proposed stopping criterion. At convergence, the learned prompt remains compact while preserving sufficient adversarial information for the downstream attack objective.

VI. CONCLUSION

In this paper, we presented ComJail, a compression-aware adversarial framework for generating compact and effective jailbreak prompts for LLMs. Motivated by the limitations of existing prompt-based jailbreak attacks in verbosity, structural

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

rigidity, and transferability, ComJail integrates contextual reconstruction and hierarchical compression into a joint generator-discriminator optimization framework. This design enables the automated synthesis of adversarial prompts that remain effective under length constraints while improving attack performance under the evaluated settings.

Extensive experiments on AdvBench and JailbreakBench across both proprietary and open-source models show that ComJail outperforms baselines under both GPT-based and human evaluation. On AdvBench-Sub, for example, ComJail achieves a 74% ASR on GPT-4, exceeding DrAttack by 11%, and reaches 91% ASR on Vicuna-7B under GPT evaluation. Additional analyses further show that ComJail preserves its advantage under fixed token budgets, remains effective in defense-aware settings, and exhibits strong source-to-target transferability. Evaluations on additional aligned models, including DeepSeek-R1 and DeepSeek-V3, further show that the proposed compression-aware strategy remains effective beyond the primary target models considered in this paper.

ComJail also produces shorter prompts than DrAttack while maintaining strong attack performance across diverse harmful intent categories. Overall, these results suggest that prompt compactness is not merely an efficiency consideration, but an important factor in effective jailbreak prompt design for modern LLMs. In future work, we will extend ComJail to multimodal settings, improve its adaptability to evolving safety mechanisms, and further investigate principled evaluation protocols for adversarial robustness and responsible red-teaming.

REFERENCES

[1] W. X. Zhao et al., "A survey of large language models," *Front. Comput. Sci.*, vol. 20, no. 12, 2026, Art. no. 2012627.

[2] J. Yang et al., "Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond," *ACM Trans. Knowl. Discov. Data*, vol. 18, pp. 1–32, 2024.

[3] Z. Hong et al., "Next-generation database interfaces: A survey of LLM-based text-to-SQL," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 12, pp. 7328–7345, 2025.

[4] Y. Zhang et al., "Siren's song in the AI ocean: A survey on hallucination in large language models," *Comput. Linguist.*, vol. 51, no. 4, pp. 1373–1418, 2025.

[5] X. Ma et al., "Safety at scale: A comprehensive survey of large model and agent safety," *Found. Trends Privacy Secur.*, vol. 8, pp. 254–469, 2025.

[6] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *Proc. High-Confidence Comput.*, vol. 4, pp. 100211–100231, 2024.

[7] S. Yi et al., "Jailbreak attacks and defenses against large language models: A survey," 2024, *arXiv:2407.04295*.

[8] Z. Xu, Y. Liu, G. Deng, Y. Li, and S. Picek, "A comprehensive study of jailbreak attack versus defense for large language models," in *Proc. Findings Assoc. Comput. Linguist.: ACL 2024*, 2024, pp. 7432–7449.

[9] K. Wang et al., "A comprehensive survey in LLM (-agent) full stack safety: Data, training and deployment," 2025, *arXiv:2504.15585*.

[10] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "HotFlip: White-box adversarial examples for text classification," in *Proc. Assoc. Comput. Linguist.* (ACL), 2018, pp. 31–36.

[11] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is BERT really robust? A strong baseline for natural language attack on text classification and entailment," in *Proc. Conf. Artif. Intell. (AAAI)*, 2020, pp. 8018–8025.

[12] L. Yuan, Y. Zhang, Y. Chen, and W. Wei, "Bridge the gap between CV and NLP! A gradient-based textual adversarial attack framework," in *Proc. Findings Assoc. Comput. Linguist.*, 2021, pp. 7132–7146.

[13] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh, "Universal adversarial triggers for attacking and analyzing NLP," in *Proc. Empirical Methods Natural Lang. Process.*, 2019, pp. 2153–2162.

[14] Y. K. Singla, S. Parekh, S. Singh, C. Chen, B. Krishnamurthy, and R. R. Shah, "Minimal: Mining models for data free universal adversarial triggers," in *Proc. Conf. Artif. Intell.*, 2022, pp. 11330–11339.

[15] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proc. Empirical Methods Natural Lang. Process.*, 2020, pp. 119–126.

[16] Y. Liu et al., "A hitchhiker's guide to jailbreaking ChatGPT via prompt engineering," in *Proc. 4th Int. Workshop Softw. Eng. AI Data Qual. Cyber-Phys. Syst./Internet Things*, 2024, pp. 12–21.

[17] N. Kandpal, E. Wallace, and C. Raffel, "Deduplicating training data mitigates privacy risks in language models," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 10697–10 707.

[18] X. Liu, N. Xu, M. Chen, and C. Xiao, "AutoDAN: Generating stealthy jailbreak prompts on aligned large language models," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1134–1154.

[19] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, *arXiv:2307.15043*.

[20] Z. Wei, Y. Wang, A. Li, Y. Mo, and Y. Wang, "Jailbreak and guard aligned language models with only few in-context demonstrations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 6, pp. 6835–6846, 2026.

[21] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, "DeepInception: Hypnotize large language model to be jailbreaker," 2023, *arXiv: 2311.03191*.

[22] H. Wang et al., "DiffusionAttacker: Diffusion-driven prompt manipulation for LLM jailbreak," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2025, pp. 22182–22194.

[23] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn.*, 2025, pp. 23–42.

[24] Y. Liu, S. Mao, X. Mei, T. Yang, and X. Zhao, "Sensitivity of adversarial perturbation in fast gradient sign method," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2019, pp. 433–436.

[25] Y. Deng and L. J. Karam, "Universal adversarial attack via enhanced projected gradient descent," in *Proc. IEEE Int. Conf. Image Process.*, 2020, pp. 1241–1245.

[26] P. Ding et al., "A wolf in Sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily," in *Proc. North Amer. Chapter Assoc. Comput. Linguist.*, 2024, pp. 2136–2153.

[27] J. Mu, X. Li, and N. Goodman, "Learning to compress prompts with gist tokens," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 19327–19352.

[28] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei, "In-context autoencoder for context compression in a large language model," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1197–1213.

[29] Z. Li, Y. Su, and N. Collier, "500XCompressor: Generalized prompt compression for large language models," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguist.*, vol. 1, 2025, pp. 25081–25091.

[30] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting language models to compress contexts," in *Proc. Empirical Methods Natural Lang. Process.*, 2023, pp. 3829–3846.

[31] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing context to enhance inference efficiency of large language models," in *Proc. Empirical Methods Natural Lang. Process.*, 2023, pp. 6342–6353.

[32] Z. Pan et al., "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Proc. Findings Assoc. Comput. Linguist.*, 2024, pp. 963–981.

[33] B. Liskavets, M. Ushakov, S. Roy, M. Klibanov, A. Etemad, and S. K. Luke, "Prompt compression with context-aware sentence encoding for fast and improved LLM inference," in *Proc. Conf. Artif. Intell.*, 2025, pp. 24595–24604.

[34] J. Liu, L. Li, T. Xiang, B. Wang, and Y. Qian, "TCRA-LLM: Token compression retrieval augmented large language model for inference cost reduction," in *Proc. Findings Empirical Methods Natural Lang. Process.*, 2023, pp. 9796–9810.

[35] A. Mehrotra et al., "Tree of attacks: Jailbreaking black-box LLMs automatically," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 61065–61105.

[36] X. Li, R. Wang, M. Cheng, T. Zhou, and C.-J. Hsieh, "DrAttack: Prompt decomposition and reconstruction makes powerful LLMs jailbreakers," in *Proc. Findings Empirical Methods Natural Lang. Process.*, 2024, pp. 13891–13913.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.

[37] J. Geng, B. Yi, Z. Fei, T. Wu, L. Nie, and Z. Liu, "When safety detectors aren't enough: A stealthy and effective jailbreak attack on LLMs via steganographic techniques," 2025, *arXiv:2505.16765*.
[38] P. Chao et al., "Jailbreakbench: An open robustness benchmark for jailbreaking large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 55005–55029.
[39] W. Zhou et al., "EasyJailBreak: A unified framework for jailbreaking large language models," 2024, *arXiv:2403.12171*.
[40] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[41] S. Wu et al., "Benchmarking open-source large language models, GPT-4 and Claude 2 on multiple-choice questions in nephrology," *NEJM AI*, vol. 1, no. 2, 2024, Art. no. AIdbp2300092.
[42] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
[43] D. Guo et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.
[44] DeepSeek-AI et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.
[45] L. Zheng et al., "Judging LLM-as-a-Judge with MT-bench and chatbot arena," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 46595–46 623.
[46] H. Touvron et al., "LLaMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[47] H. Inan et al., "LLaMA Guard: LLM-based input-output safeguard for human-AI conversations," 2023, *arXiv:2312.06674*.
[48] G. Alon and M. Kamfonas, "Detecting language model attacks with perplexity," 2023, *arXiv:2308.14132*.
[49] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "SmoothLLM: Defending large language models against jailbreaking attacks," *Trans. Mach. Learn. Res.*, Apr. 2025.
[50] X. Wang et al., "SelfDefend: LLMs can defend themselves against jailbreaking in a practical manner," in *Proc. USENIX Secur. Symp.*, 2025, pp. 2441–2460.



Wenxiao Liu received the BS and MS degrees from Jinan University, Guangzhou, China, in 2020 and 2023, respectively, where he is currently working toward the PhD degree with the College of Cyber Security. His research interests span natural language processing, large language model security, trustworthy artificial intelligence, adversarial machine learning, prompt injection and jailbreak defense, privacy-preserving machine learning, graph neural networks, and their applications to cybersecurity.



Zhiquan Liu (Senior Member, IEEE) received the BS degree from the School of Science, Xidian University, Xi'an, China, in 2012, and the PhD degree from the School of Computer Science and Technology, Xidian University, Xi'an, China, in 2017. He is currently a full professor, doctoral supervisor, and the deputy dean with the College of Cyber Security, Jinan University, Guangzhou, China. His research interests include security, trust, privacy, and intelligence in vehicular networks and UAV networks. He is also the area editor or associate editor for multiple SCI-index journals, such as *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Vehicular Technology*, *IEEE Internet of Things Journal*, *IEEE Network*, and *Information Fusion*. His homepage is https://www.zqliu.com.



Chong Xu received the BS degree from the School of Software Engineering, Henan University of Science and Technology, Luoyang, China, in 2022. He is currently working toward the Graduate degree with the College of Cyber Security, Jinan University, Guangzhou, China. He has authored or coauthored at the 2nd International Conference on AI-Generated Content (AIGC), where he was the recipient of the Best Paper Award. His research interests include large language models and large language model security.



Chaozhuo Li received the PhD degree in engineering from Beihang University, Beijing, China, in 2020. From 2017 to 2019, he participated in a joint training program with the University of Illinois, USA. From 2020 to 2024, he was a Senior Researcher with Microsoft Research Asia. He is currently an associate researcher and distinguished scholar with the Beijing University of Posts and Telecommunications, Beijing, China. His research interests include large language model security, natural language processing, and social network analysis.



Jingjing Guo received the MSc and PhD degrees from the School of Computer Science and Technology, Xidian University, Xi'an, China, in 2012 and 2015, respectively. She is currently an associate professor with the School of Cyber Engineering, Xidian University, Xi'an, China. Her research interests include trust management, access control, Internet of Things security, and artificial intelligence security.



Jianfeng Ma received the ME degree from the School of Computer Science and Technology, Xidian University, Xi'an, China, in 1988, and the PhD degree from the School of Information and Telecommunication Engineering, Xidian University, Xi'an, China, in 1995. He is currently a professor with the School of Cyber Engineering, Xidian University, Xi'an, China. His research interests include information security, coding theory, and cryptography.



Feiran Huang (Senior Member, IEEE) received the PhD degree from the School of Computer Science and Engineering, Beihang University, Beijing, China, in 2019. He is currently a professor with the School of Computer Science and Engineering, Beihang University. He has more than 100 publications appearing in top conferences and journals, such as SIGIR, KDD, WWW, MM, *IEEE Transactions on Image Processing*, IJCV, and *IEEE Transactions on Knowledge and Data Engineering*. He was the recipient of the five Best Paper awards, such as the Best Paper Award Honourable Mention in SIGIR 2023 and Best Paper Award Runner Up in PAKDD 2023. He holds more than 50 Chinese, U.S., and International Granted Patents. He is also an Associate Editor for *IEEE Transactions on Affective Computing* and Elsevier CAEE. His research interests include recommender systems, social networks, sentiment analysis, and large language models.

Authorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on October 16,2026 at 11:58:18 UTC from IEEE Xplore. Restrictions apply.